

TUTORIAL INTERACTIVO · GUÍA DEL ESTUDIANTE · NIVEL
FUNDAMENTOS A AVANZADO

BACKEND

NODE · EXPRESS · TYPESCRIPT

P A S O A P A S O

♦ 21 lecciones · 42 retos prácticos · API REST completa

¿Qué es Node.js? · ¿Qué es una API REST?

Métodos HTTP · Códigos de estado · Express + TypeScript

CRUD · Validación · Errores · Tests · Despliegue

MATERIAL DIDÁCTICO · DESARROLLO DE SOFTWARE · 2026

Construye, prueba y despliega una API REST profesional desde cero, con el código completo de cada archivo



LECCIÓN 00

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

RUTA DE ESTUDIO Y CÓMO USAR LA GUÍA

Tu mapa para construir un backend profesional desde cero

Bienvenido al tutorial de desarrollo backend con Node.js, Express y TypeScript. Esta guía te lleva desde los conceptos más básicos (qué es Node.js, qué es una API REST) hasta construir, probar y desplegar en producción una API REST completa y profesional. No necesitas experiencia previa en backend: solo conocimientos básicos de JavaScript. Al terminar, tendrás una API funcional desplegada en internet y el conocimiento para construir las tuyas.

El tutorial está construido alrededor de un **proyecto real que crece lección a lección**: una API de gestión de tareas (un "To-Do" profesional). Cada lección agrega una pieza al proyecto, y al final tendrás todo el código completo, probado y en producción. Te recomendamos teclear cada archivo tú mismo en lugar de copiar y pegar: así aprendes de verdad.



ESTRUCTURA DE CADA LECCIÓN

	CONCEPTO Definición breve del tema de la lección. Léelo primero.
	EXPLICACIÓN El tema en profundidad, con contexto y el porqué de cada decisión.
	CÓDIGO / TERMINAL Código completo de cada archivo y los comandos exactos a ejecutar.
	TIPS DEL EXPERTO Consejos de la práctica profesional que evitan errores comunes.
	PRACTICA Dos retos por lección para afianzar lo aprendido.



RUTA DE APRENDIZAJE

Las 21 lecciones se agrupan en cuatro módulos que van de la teoría a la práctica y de lo simple a producción:

MÓDULO 1 — Fundamentos teóricos

- Lección 01 — ¿Qué es Node.js?
- Lección 02 — ¿Qué es una API REST?
- Lección 03 — Métodos HTTP (GET, POST, PUT, PATCH, DELETE)
- Lección 04 — Códigos de estado HTTP (2xx, 3xx, 4xx, 5xx)

MÓDULO 2 — Configuración del proyecto

- Lección 05 — Preparar el entorno (Node, npm, TypeScript)
- Lección 06 — Inicializar el proyecto (package.json + tsconfig)
- Lección 07 — Tu primer servidor Express + TypeScript
- Lección 08 — Estructura de carpetas profesional

MÓDULO 3 — Construir la API

- Lección 09 — Rutas y controladores
- Lección 10 — Middlewares
- Lección 11 — Modelo de datos y capa de servicio
- Lección 12 — CRUD completo
- Lección 13 — Validación de datos
- Lección 14 — Manejo de errores centralizado
- Lección 15 — Variables de entorno y configuración
- Lección 16 — Persistencia de datos

MÓDULO 4 — Probar y desplegar

- Lección 17 — Probar la API (Thunder Client, Postman, curl)
- Lección 18 — Tests automatizados (Jest + Supertest)
- Lección 19 — Build y preparación para producción
- Lección 20 — Desplegar en producción

REQUISITOS PREVIOS

Solo necesitas: (1) conocimientos básicos de JavaScript (variables, funciones, objetos, arrays), (2) una computadora con Windows, macOS o Linux, y (3) ganas de aprender. Todo lo demás —Node.js, TypeScript, Express— lo instalaremos y explicaremos paso a paso.



LECCIÓN 01

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

¿QUÉ ES NODE.JS?

JavaScript fuera del navegador, en el servidor

CONCEPTO

Node.js es un entorno de ejecución que permite correr código JavaScript **fuera del navegador**, directamente en tu computadora o en un servidor. Está construido sobre el motor **V8** de Google Chrome (el mismo que ejecuta JS en el navegador) y le agrega la capacidad de acceder al sistema de archivos, la red, bases de datos y el sistema operativo. Con Node.js, JavaScript deja de ser solo un lenguaje de páginas web y se convierte en una herramienta para construir servidores, APIs, herramientas de línea de comandos y más.



EXPLICACIÓN DETALLADA

Antes de Node.js (2009), JavaScript solo vivía en el navegador: servía para dar interactividad a las páginas web. Si querías programar la parte del servidor (el "backend"), tenías que usar otros lenguajes como PHP, Java, Python o Ruby. Node.js cambió eso al tomar el motor V8 —que ya compilaba JavaScript a código máquina muy rápido— y empaquetarlo para funcionar de forma independiente, con acceso a recursos del sistema. El resultado: puedes usar **el mismo lenguaje** en el navegador y en el servidor.

La característica técnica más importante de Node.js es que es **asíncrono y orientado a eventos**, con un modelo de un solo hilo (single-threaded) basado en un "event loop". En lugar de bloquear el programa esperando que termine una operación lenta (leer un archivo, consultar una base de datos, responder una petición de red), Node.js registra una función de retorno (callback/promesa) y sigue atendiendo otras tareas. Cuando la operación termina, ejecuta el callback. Esto lo hace excelente para manejar miles de conexiones simultáneas sin consumir muchos recursos.

Node.js viene con **npm** (Node Package Manager), el gestor de paquetes más grande del mundo. A través de npm puedes instalar bibliotecas de terceros (como Express, que usaremos en este tutorial) con un solo comando. El archivo `package.json` describe tu proyecto y sus dependencias. Esta combinación de un runtime potente y un ecosistema gigantesco de paquetes reutilizables es lo que hizo a Node.js tan popular para el desarrollo backend moderno.

Node.js es ideal para construir **APIs REST**, servidores web, aplicaciones en tiempo real (chats, juegos), microservicios y herramientas de desarrollo. Empresas como Netflix, LinkedIn, PayPal, Uber y NASA lo usan en producción. No es la mejor opción para tareas de cómputo intensivo y

prolongado (como procesamiento de video o cálculos científicos pesados), porque su modelo de un solo hilo se bloquearía; para eso existen otras herramientas. Pero para el tipo de trabajo que hace un backend web —recibir peticiones, consultar datos, responder— es una de las mejores opciones disponibles.



EJEMPLO PRÁCTICO

 **Caso:** Verificar que entiendes el modelo asíncrono de Node.js con un ejemplo mínimo que puedes correr apenas lo instales (lección 05).

hola-node.js

```
// hola-node.js – Tu primer contacto con Node.js

console.log('1. Inicio del programa');

// setTimeout NO bloquea: registra el callback y sigue
setTimeout(() => {
  console.log('3. Esto aparece 2 segundos después');
}, 2000);

console.log('2. Fin del archivo (pero el programa sigue vivo)');

// Node.js mantiene el proceso vivo hasta que
// no queden tareas pendientes en el event loop
```

SALIDA EN CONSOLA

1. Inicio del programa
2. Fin del archivo (pero el programa sigue vivo)
3. Esto aparece 2 segundos después

EL ORDEN IMPORTA

Fíjate que el mensaje "3" aparece **al final**, aunque en el código está en el medio. Eso es el modelo asíncrono en acción: Node.js no se queda esperando los 2 segundos, sino que continúa con la línea siguiente y ejecuta el callback cuando el temporizador termina. Entender esto es la base de todo el desarrollo en Node.

★ TIPS DEL EXPERTO

- ✦ Node.js usa **JavaScript**, pero en este tutorial usaremos **TypeScript** (JavaScript con tipos), que se compila a JavaScript y previene muchísimos errores antes de ejecutar.
- ✦ El **event loop** es el corazón de Node. No necesitas dominarlo a fondo todavía, pero recuerda: las operaciones lentas (red, disco, BD) no bloquean; usan callbacks, promesas o [async/await](#).
- ✦ Verifica tu versión con `node --version`. Para este tutorial usa **Node 20 LTS** o superior: las versiones LTS (Long Term Support) son las más estables para producción.
- ✦ Node.js no es un lenguaje ni un framework: es un **entorno de ejecución**. Express (que veremos pronto) sí es un framework que corre sobre Node.

🚀 PRACTICA LO APRENDIDO



RETO 1

Investiga el ecosistema

Visita el sitio oficial nodejs.org e identifica: cuál es la versión LTS actual, cuál es la versión "Current", y qué diferencia hay entre ambas. Anota por qué para producción se recomienda siempre la LTS.



RETO 2

Predice la salida

Sin ejecutarlo, escribe en papel qué orden tendrán estos `console.log`: uno normal al inicio, un `setTimeout` de 0 ms en el medio, y otro `console.log` normal al final. Luego (tras instalar Node en la lección 05) ejecútalo y comprueba si acertaste. ¿Por qué el `setTimeout(0)` no sale segundo?



LECCIÓN 02

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

¿QUÉ ES UNA API REST?

El estándar para que aplicaciones se comuniquen por internet

CONCEPTO

Una **API** (Application Programming Interface) es un conjunto de reglas que permite que dos programas se comuniquen entre sí. Una **API REST** (REpresentational State Transfer) es un estilo de arquitectura para diseñar APIs que funcionan sobre HTTP, el mismo protocolo de la web. En una API REST, todo gira en torno a **recursos** (usuarios, productos, tareas) identificados por **URLs**, y se manipulan usando los **métodos HTTP** (GET, POST, PUT, DELETE). Es el estándar más usado para construir backends web modernos.



EXPLICACIÓN DETALLADA

Imagina un restaurante: tú (el cliente) no entras a la cocina a preparar tu comida. Le pides al mesero (la API) lo que quieres usando un lenguaje común (el menú), y el mesero se comunica con la cocina (el servidor/base de datos) y te trae el resultado. Una API REST es ese mesero: una interfaz clara y estandarizada entre quien pide datos (una app móvil, una web, otro servidor) y quien los tiene (tu backend). El cliente no necesita saber cómo está hecha la cocina, solo cómo pedir.

REST se basa en el concepto de **recursos**. Un recurso es cualquier cosa que tu aplicación maneja: una tarea, un usuario, un producto. Cada recurso tiene una **URL única** que lo identifica. Por convención, las URLs usan sustantivos en plural: `/tareas` representa la colección de todas las tareas, y `/tareas/42` representa la tarea con id 42. Sobre estas URLs aplicas métodos HTTP para leer, crear, actualizar o borrar. Esta combinación de URL + método define exactamente qué operación se realiza.

Las APIs REST son **sin estado (stateless)**: cada petición del cliente contiene toda la información necesaria para procesarla, y el servidor no guarda memoria de peticiones anteriores. Si necesitas identificar al usuario, esa información (un token de autenticación, por ejemplo) viaja en cada petición. Esto hace que las APIs REST sean fáciles de escalar: como el servidor no recuerda nada entre peticiones, puedes tener muchas copias del servidor atendiendo en paralelo sin problemas de sincronización.

El formato de datos más común en las APIs REST modernas es **JSON** (JavaScript Object Notation), un formato de texto ligero y legible que representa objetos con pares clave-valor. Cuando pides una tarea, el servidor te responde con un JSON como `{ "id": 42, "titulo": "Estudiar",`

"completada": false }. JSON es independiente del lenguaje: tu backend puede estar en Node.js y el cliente en Swift (iOS), Kotlin (Android) o React (web), y todos entienden JSON perfectamente. Esa interoperabilidad es una de las grandes ventajas de REST.

EJEMPLO PRÁCTICO

 **Caso:** Ver cómo se vería una conversación típica entre un cliente y una API REST de tareas.

 **ejemplo-conversacion-rest.http**

```
// === PETICIÓN del cliente al servidor ===
// EL cliente quiere la lista de todas las tareas
GET /api/tareas HTTP/1.1
Host: miapi.com
Accept: application/json

// === RESPUESTA del servidor al cliente ===
HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 1, "titulo": "Aprender Node.js", "completada": true },
  { "id": 2, "titulo": "Construir una API", "completada": false }
]

// === Crear una tarea nueva ===
POST /api/tareas HTTP/1.1
Content-Type: application/json

{ "titulo": "Desplegar a producción" }

// EL servidor responde con la tarea creada y su nuevo id
HTTP/1.1 201 Created
{ "id": 3, "titulo": "Desplegar a producción", "completada": false }
```

ANATOMÍA DE UN ENDPOINT REST

Operación	Método + URL	Significado
Listar todas	GET /tareas	Obtener la colección completa
Obtener una	GET /tareas/5	Obtener la tarea con id 5
Crear	POST /tareas	Crear una tarea nueva

Operación	Método + URL	Significado
Reemplazar	PUT /tareas/5	Actualizar por completo la tarea 5
Modificar	PATCH /tareas/5	Actualizar parcialmente la tarea 5
Borrar	DELETE /tareas/5	Eliminar la tarea 5

★ TIPS DEL EXPERTO

- ✦ Usa **sustantivos en plural** para tus recursos: [/tareas](#), [/usuarios](#), [/productos](#). Nunca verbos como [/crearTarea](#): el verbo lo da el método HTTP.
- ✦ Una API REST bien diseñada es **predecible**: si conoces [/tareas](#), ya intuyes que [/usuarios](#) funcionará igual. La consistencia es la clave de una buena API.
- ✦ REST trabaja con **JSON** casi siempre. Asegúrate de enviar el header **Content-Type: application/json** cuando mandas datos en el cuerpo.
- ✦ No confundas REST con un protocolo: REST es un **estilo de diseño**. HTTP es el protocolo. La mayoría de APIs REST usan HTTP, pero REST es un conjunto de buenas prácticas, no una ley estricta.

🚀 PRACTICA LO APRENDIDO



RETO 1

Diseña los endpoints

Imagina una API REST para una biblioteca. Diseña los endpoints (método + URL) para: listar todos los libros, obtener un libro específico, agregar un libro nuevo, actualizar la información de un libro y eliminar un libro. Escríbelos siguiendo las convenciones REST.



RETO 2

Identifica recursos

Para una app de red social, identifica al menos 4 recursos (sustantivos) que tendría su API y, para cada uno, escribe cómo se vería la URL para listar la colección y para obtener un elemento individual. Piensa en relaciones: ¿cómo listarías los comentarios de una publicación específica?



LECCIÓN 03

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

MÉTODOS HTTP

Los verbos que indican qué acción realizar sobre un recurso

CONCEPTO

Los **métodos HTTP** (también llamados "verbos") indican qué tipo de acción quiere realizar el cliente sobre un recurso. Los cinco más usados en APIs REST son: **GET** (leer), **POST** (crear), **PUT** (reemplazar), **PATCH** (modificar parcialmente) y **DELETE** (borrar). La combinación de un método con una URL define una operación completa: **GET** `/tareas` lista tareas, mientras que **DELETE** `/tareas/5` borra la tarea 5. El método le dice al servidor la **intención**; la URL le dice **sobre qué**.



EXPLICACIÓN DETALLADA

GET sirve para **leer** datos sin modificarlos. Es el método más común: cuando abres una página web o pides una lista de productos, usas GET. Una característica clave es que GET debe ser **seguro** (no cambia nada en el servidor) e **idempotente** (llamarlo una o mil veces produce el mismo resultado). Por eso los parámetros de GET van en la URL (como `/tareas?completada=true`), nunca en el cuerpo: las peticiones GET típicamente no llevan cuerpo (body).

POST sirve para **crear** un recurso nuevo. Cuando envías un formulario de registro o creas una tarea, usas POST. Los datos del nuevo recurso van en el **cuerpo (body)** de la petición, normalmente en formato JSON. POST **no es idempotente**: si lo llamas dos veces, creas dos recursos distintos. Por eso, tras un POST exitoso, el servidor suele responder con el código **201 Created** y devuelve el recurso recién creado, incluyendo el **id** que le asignó.

PUT y **PATCH** ambos **actualizan** un recurso existente, pero con una diferencia importante. **PUT reemplaza el recurso completo**: debes enviar todos los campos, y los que omitas se borran o quedan vacíos. **PATCH modifica solo los campos que envías**, dejando el resto intacto. Por ejemplo, para marcar una tarea como completada sin tocar su título, PATCH es ideal: envías solo `{ "completada": true }`. Ambos son idempotentes: aplicar la misma actualización varias veces deja el recurso en el mismo estado.

DELETE elimina un recurso. **DELETE** `/tareas/5` borra la tarea 5. Es idempotente en el sentido de que borrar algo ya borrado deja el sistema en el mismo estado (aunque la segunda llamada suele devolver 404 porque el recurso ya no existe). Existen otros métodos menos comunes como **HEAD** (como GET pero sin cuerpo, solo headers) y **OPTIONS** (consultar qué métodos permite un recurso,

usado en CORS), pero con los cinco principales cubres la inmensa mayoría de las necesidades de una API REST.

RESUMEN DE LOS MÉTODOS HTTP

Método	Acción	¿Cuerpo?	Idempotente
GET	Leer / consultar	No	Sí
POST	Crear nuevo	Sí	No
PUT	Reemplazar todo	Sí	Sí
PATCH	Modificar parcial	Sí	Sí
DELETE	Eliminar	No	Sí

EJEMPLO PRÁCTICO

 **Caso:** Ver cómo cada método actúa sobre el mismo recurso /tareas en una API REST.

metodos-http-en-accion.http

```
// GET - Leer todas las tareas (sin cuerpo)
GET /api/tareas
200 OK [ {id:1,...}, {id:2,...} ]

// GET - Leer una tarea específica
GET /api/tareas/1
200 OK { id:1, titulo:'Estudiar', completada:false }

// POST - Crear una tarea (datos en el cuerpo)
POST /api/tareas
Body: { "titulo": "Hacer ejercicio" }
201 Created { id:3, titulo:'Hacer ejercicio', completada:false }

// PUT - Reemplazar la tarea 3 por completo
PUT /api/tareas/3
Body: { "titulo": "Correr 5km", "completada": true }
200 OK { id:3, titulo:'Correr 5km', completada:true }

// PATCH - Solo cambiar el estado, sin tocar el título
PATCH /api/tareas/3
Body: { "completada": false }
200 OK { id:3, titulo:'Correr 5km', completada:false }

// DELETE - Borrar la tarea 3
```

```
DELETE /api/tareas/3
204 No Content (sin cuerpo)
```

★ TIPS DEL EXPERTO

- ✦ Usa **PATCH** cuando solo quieras cambiar uno o pocos campos. Usar PUT te obligaría a reenviar el objeto completo, arriesgando borrar datos por accidente.
- ✦ **GET nunca debe modificar datos.** Si una operación cambia algo en el servidor, usa POST, PUT, PATCH o DELETE. Un GET que borra cosas es un anti-patrón peligroso.
- ✦ Tras un **POST** exitoso, responde **201 Created** y devuelve el recurso con su nuevo **id**. El cliente lo necesita para futuras operaciones.
- ✦ Los navegadores solo hacen GET y POST desde formularios HTML. Para PUT, PATCH y DELETE se usa JavaScript (**fetch**) o herramientas como Postman. Tu API igual debe soportarlos todos.

🚀 PRACTICA LO APRENDIDO



RETO 1

Asigna el método correcto

Para cada acción, indica qué método HTTP usarías y la URL: (a) ver el perfil del usuario 7, (b) registrar un usuario nuevo, (c) cambiar solo el email del usuario 7, (d) eliminar la cuenta del usuario 7, (e) reemplazar todos los datos del usuario 7. Justifica por qué eliges PUT o PATCH en los casos de actualización.



RETO 2

Idempotencia en la práctica

Explica con tus palabras por qué POST no es idempotente pero PUT sí. Da un ejemplo concreto: ¿qué pasaría si tu conexión falla y reenvías por error un POST de "crear pedido"? ¿Y si reenvías un PUT de "actualizar dirección"? ¿Cuál es más peligroso reenviar y por qué?



LECCIÓN 04

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

CÓDIGOS DE ESTADO HTTP

El número que dice si la petición salió bien o mal

CONCEPTO

Cada respuesta HTTP incluye un **código de estado**: un número de tres dígitos que resume qué pasó con la petición. Se agrupan en cinco familias según su primer dígito: **1xx** (informativos), **2xx** (éxito), **3xx** (redirecciones), **4xx** (error del cliente) y **5xx** (error del servidor). Usar los códigos correctos es fundamental: le permiten al cliente saber, de un vistazo, si debe procesar los datos, reintentar, corregir su petición o reportar un fallo del servidor.



EXPLICACIÓN DETALLADA

Los códigos **2xx** indican **éxito**. El más común es **200 OK**, que significa "todo salió bien" y se usa en respuestas GET, PUT y PATCH exitosas. **201 Created** se usa tras crear un recurso con POST. **204 No Content** indica éxito pero sin datos que devolver, perfecto para un DELETE exitoso (borraste algo, no hay nada que mostrar). Devolver el 2xx correcto le comunica al cliente el resultado exacto de su operación.

Los códigos **4xx** indican que **el cliente cometió un error** en su petición, y el servidor no puede o no debe procesarla. **400 Bad Request** significa que la petición está mal formada (faltan campos, JSON inválido). **401 Unauthorized** indica que el cliente no se ha autenticado (no envió credenciales válidas). **403 Forbidden** significa que está autenticado pero no tiene permiso para esa acción. **404 Not Found**, el más famoso, indica que el recurso solicitado no existe. **409 Conflict** señala un conflicto, como intentar registrar un email que ya existe.

Los códigos **5xx** indican que **el servidor falló** al procesar una petición que era válida. **500 Internal Server Error** es el genérico: algo se rompió en tu código (una excepción no controlada, una división por cero, un error de conexión a la base de datos). **503 Service Unavailable** indica que el servidor está temporalmente sobrecargado o en mantenimiento. La diferencia clave con los 4xx es la **culpa**: en 4xx el cliente debe corregir su petición; en 5xx el cliente no puede hacer nada, el problema está en el servidor.

Los códigos **3xx** son **redirecciones**: le dicen al cliente que el recurso está en otra ubicación. **301 Moved Permanently** (movido para siempre) y **302 Found** (movido temporalmente) son los más comunes, usados sobre todo en navegación web. **304 Not Modified** se usa con caché: indica que el recurso no cambió desde la última vez, así el cliente usa su copia guardada. En APIs REST los

3xx se usan menos que los 2xx y 4xx, pero es bueno conocerlos. La familia **1xx** (informativos) es poco frecuente en el desarrollo cotidiano.

CÓDIGOS QUE USARÁS A DIARIO

Código	Nombre	Cuándo usarlo
200	OK	GET, PUT o PATCH exitoso
201	Created	POST que creó un recurso
204	No Content	DELETE exitoso, sin cuerpo
400	Bad Request	Datos inválidos o mal formados
401	Unauthorized	Falta autenticación
403	Forbidden	Autenticado pero sin permiso
404	Not Found	El recurso no existe
409	Conflict	Conflicto (ej. email duplicado)
500	Server Error	Error inesperado del servidor

EJEMPLO PRÁCTICO

 **Caso:** Ver qué código devolver en distintos escenarios de la API de tareas.

codigos-en-accion.http

```
// ✔ Petición exitosa de lectura
GET /api/tareas/1      200 OK
{ id:1, titulo:'Estudiar', completada:false }

// ✔ Creación exitosa
POST /api/tareas      201 Created
{ id:5, titulo:'Nueva tarea', completada:false }

// ✔ Borrado exitoso (sin cuerpo)
DELETE /api/tareas/5   204 No Content

// ✖ El cliente pide una tarea que no existe
GET /api/tareas/999    404 Not Found
{ error: 'Tarea con id 999 no encontrada' }
```

```
// ❌ El cliente envía datos inválidos (título vacío)
POST /api/tareas      400 Bad Request
Body: { "titulo": "" }
{ error: 'El título es obligatorio' }

// ❌ Algo se rompió en el servidor
GET /api/tareas      500 Internal Server Error
{ error: 'Error interno del servidor' }
```

⚠️ REGLA DE ORO

Si la culpa es del **cliente** (datos malos, recurso inexistente, sin permisos), usa un código **4xx**. Si la culpa es del **servidor** (una excepción, un bug, la base de datos caída), usa un código **5xx**. Nunca devuelvas 200 con un mensaje de error dentro: el código de estado debe reflejar la realidad de lo que pasó.

★ TIPS DEL EXPERTO

- ✦ No devuelvas **200 OK** para los errores. Un cliente que recibe 200 asume que todo salió bien y procesará la respuesta como datos válidos.
- ✦ Acostúmbrate a los cuatro que más usarás: **200** (todo bien), **201** (creado), **404** (no existe), **400** (datos malos). Con esos cubres el 80% de los casos.
- ✦ Para errores, devuelve siempre un **cuerpo JSON descriptivo** además del código: `{ "error": "mensaje claro" }`. Ayuda muchísimo a depurar del lado del cliente.
- ✦ El **500** casi siempre significa un bug en tu código. En producción, registra (log) el error completo del lado del servidor, pero nunca expongas detalles internos (stack traces) al cliente por seguridad.

🚀 PRACTICA LO APRENDIDO



RETO 1

Asigna el código correcto

Indica qué código de estado HTTP devolverías en cada caso: (a) un usuario consulta su lista de pedidos exitosamente, (b) intenta acceder a un pedido que no existe, (c) crea un pedido nuevo correctamente, (d) intenta crear un pedido sin especificar productos, (e) la base de datos está caída y la consulta falla.



Diseña los errores de un login

RETO 2

Para un endpoint `POST /login`, define qué código y mensaje devolverías cuando: (a) el login es exitoso, (b) el usuario no envía email ni contraseña, (c) las credenciales son incorrectas, (d) la cuenta existe pero está bloqueada por permisos. Escribe el código + el cuerpo JSON de respuesta para cada caso.



LECCIÓN 05

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

PREPARAR EL ENTORNO

Instalar Node.js, npm y TypeScript en tu máquina



CONCEPTO

Antes de escribir código necesitas tres herramientas: **Node.js** (el entorno de ejecución), **npm** (el gestor de paquetes, que viene incluido con Node) y **TypeScript** (el lenguaje que usaremos, que añade tipos a JavaScript). También necesitarás un editor de código; recomendamos **Visual Studio Code** por su excelente soporte para TypeScript. Esta lección te guía en la instalación y verificación de cada pieza.



EXPLICACIÓN DETALLADA

Node.js se descarga desde el sitio oficial nodejs.org. Siempre elige la versión **LTS** (Long Term Support), que es la más estable y recibe soporte por años. El instalador incluye automáticamente **npm**, así que con una sola instalación obtienes ambas herramientas. En Windows y macOS el instalador es gráfico (siguiente, siguiente, listo); en Linux puedes usar el gestor de paquetes de tu distribución o, mejor aún, un gestor de versiones como **nvm**.

Te recomendamos instalar **nvm** (Node Version Manager) en lugar de instalar Node directamente. **nvm** te permite tener varias versiones de Node instaladas y cambiar entre ellas con un comando. Esto es muy útil cuando trabajas en varios proyectos que requieren versiones distintas. En macOS y Linux se instala con un script; en Windows existe [nvm-windows](#). Si recién empiezas y quieres simplicidad, el instalador oficial está perfectamente bien.

TypeScript es un superconjunto de JavaScript creado por Microsoft que añade un sistema de **tipos estáticos**. Esto significa que declaras de qué tipo es cada variable, parámetro y retorno, y el compilador verifica que los uses correctamente **antes** de ejecutar el programa. TypeScript no corre directamente: se **compila** (transpila) a JavaScript normal, que es lo que Node.js ejecuta. Para desarrollo usaremos herramientas que hacen esta compilación automáticamente, así que no notarás el paso intermedio.

Usamos TypeScript en lugar de JavaScript puro porque en proyectos de backend los tipos previenen una enorme cantidad de errores: te avisan si pasas un número donde se esperaba un texto, si accedes a una propiedad que no existe, o si olvidas manejar un caso. En un equipo de desarrollo, los tipos sirven también como documentación viva del código. La inversión inicial de aprender la sintaxis de tipos se paga con creces en menos bugs y código más mantenible.



EJEMPLO PRÁCTICO



Caso: Instalar y verificar todas las herramientas necesarias paso a paso.



TERMINAL

```
# 1. Verifica si ya tienes Node.js instalado
$ node --version
v20.11.1

# 2. Verifica npm (viene con Node)
$ npm --version
10.2.4

# Si Los comandos anteriores fallan, instala Node LTS
# desde https://nodejs.org (elige el botón 'LTS')

# 3. Instala TypeScript de forma global (opcional)
$ npm install -g typescript

# 4. Verifica la instalación de TypeScript
$ tsc --version
Version 5.4.5

# 5. Instala ts-node-dev para desarrollo (lo usaremos por proyecto)
# No Lo instales global; Lo haremos por proyecto en La Lección 06
```

✓ ¿GLOBAL O POR PROYECTO?

Instalar TypeScript de forma global (-g) es opcional y cómodo para probar. Pero en proyectos reales se instala **por proyecto** (como dependencia de desarrollo), para que todo el equipo use exactamente la misma versión. En la próxima lección lo haremos así. Si solo quieres probar **tsc** rápido, la instalación global está bien.

★ TIPS DEL EXPERTO

- ✦ Elige siempre la versión **LTS** de Node para proyectos serios. Las versiones "Current" traen lo último pero pueden ser inestables.
- ✦ Instala **Visual Studio Code** (gratis, de Microsoft). Su soporte de TypeScript es el mejor: autocompletado, detección de errores en tiempo real y refactorizaciones.

- ✦ En VS Code, instala las extensiones **ESLint** y **Prettier**: la primera detecta problemas de código, la segunda formatea automáticamente. Son estándar en la industria.
- ✦ Si trabajas en varios proyectos, aprende **nvm**. Poder hacer `nvm use 18` o `nvm use 20` según el proyecto te ahorrará muchos dolores de cabeza.

PRACTICA LO APRENDIDO



RETO 1

Instala y verifica

Instala Node.js LTS en tu máquina (si no lo tienes). Ejecuta `node --version` y `npm --version` y anota las versiones. Luego abre el REPL de Node escribiendo solo `node` en la terminal, ejecuta `2 + 2` y `console.log('Hola Node')`, y sal con `.exit`. Familiarízate con el entorno.



RETO 2

Compila tu primer TypeScript

Crea un archivo `prueba.ts` con: `const mensaje: string = 'Hola TypeScript'; console.log(mensaje);`. Compíllalo con `tsc prueba.ts` (genera `prueba.js`) y ejecútalo con `node prueba.js`. Luego, cambia el tipo a `number` sin cambiar el valor `string` y observa el error que TypeScript te muestra al compilar.



LECCIÓN 06

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

INICIALIZAR EL PROYECTO

Crear package.json, instalar Express y configurar TypeScript CONCEPTO

Todo proyecto de Node.js empieza con un archivo `package.json`: el documento que describe tu proyecto, lista sus dependencias y define scripts para tareas comunes. En esta lección crearemos el proyecto, instalaremos **Express** (el framework web) y las herramientas de **TypeScript**, y configuraremos el compilador con un archivo `tsconfig.json`. Al terminar tendrás el esqueleto listo para escribir el servidor.

 EXPLICACIÓN DETALLADA

El comando `npm init -y` crea un `package.json` con valores por defecto. Este archivo es el centro de control del proyecto: registra el nombre, la versión, el punto de entrada, los scripts y —lo más importante— las **dependencias**. Cuando instalas un paquete con `npm install`, npm lo descarga a la carpeta `node_modules` y lo anota en `package.json`. Así, cualquier persona que clone tu proyecto puede instalar todo con un solo `npm install`.

Hay dos tipos de dependencias. Las **dependencias de producción** (`dependencies`) son las que tu aplicación necesita para funcionar en producción, como Express. Las **dependencias de desarrollo** (`devDependencies`, instaladas con `--save-dev` o `-D`) son herramientas que solo necesitas mientras programas: el compilador de TypeScript, los tipos, las herramientas de recarga automática. Esta separación mantiene tu despliegue de producción ligero.

Para usar Express con TypeScript necesitas instalar **dos cosas por cada librería**: la librería en sí (`express`) y sus **tipos** (`@types/express`). Los paquetes `@types/...` son archivos de declaración que le dicen a TypeScript cómo está tipada esa librería, habilitando el autocompletado y la verificación. Muchas librerías modernas ya incluyen sus propios tipos, pero Express y Node requieren los paquetes `@types` por separado.

El archivo `tsconfig.json` configura cómo TypeScript compila tu código: qué versión de JavaScript generar (`target`), dónde están los archivos fuente (`rootDir`), dónde poner el JavaScript compilado (`outDir`), y decenas de opciones de rigor. Activaremos `strict: true`, que habilita todas las comprobaciones estrictas de tipos: es más exigente al principio, pero te obliga a escribir código más seguro y es el estándar profesional.

EJEMPLO PRÁCTICO

 **Caso:** Crear el proyecto desde cero, instalar dependencias y configurar TypeScript.

TERMINAL

```
# 1. Crea la carpeta del proyecto y entra en ella
$ mkdir api-tareas && cd api-tareas

# 2. Inicializa el package.json con valores por defecto
$ npm init -y

# 3. Instala Express (dependencia de producción)
$ npm install express

# 4. Instala TypeScript y los tipos (dependencias de desarrollo)
$ npm install -D typescript @types/express @types/node

# 5. Instala ts-node-dev para recarga automática en desarrollo
$ npm install -D ts-node-dev

# 6. Genera un tsconfig.json base
$ npx tsc --init

# Tu proyecto ahora tiene node_modules, package.json y tsconfig.json
```

EL ARCHIVO package.json

package.json

```
{
  "name": "api-tareas",
  "version": "1.0.0",
  "description": "API REST de tareas con Express y TypeScript",
  "main": "dist/server.js",
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/server.ts",
    "build": "tsc",
    "start": "node dist/server.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "@types/express": "^4.17.21",
    "@types/node": "^20.11.30",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.4.5"
  }
}
```

EL ARCHIVO tsconfig.json

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",           // versión de JS a generar
    "module": "commonjs",        // sistema de módulos de Node
    "rootDir": "./src",          // dónde está el código fuente
    "outDir": "./dist",          // dónde poner el JS compilado
    "strict": true,              // todas las comprobaciones estrictas
    "esModuleInterop": true,     // interoperabilidad de imports
    "skipLibCheck": true,        // no revisar tipos de node_modules
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,   // permite importar archivos .json
    "sourceMap": true            // mapas para depurar el TS original
  },
  "include": ["src/**/*.ts"],    // qué archivos compilar
  "exclude": ["node_modules", "dist"]
}
```

⚠ AÑADE UN .gitignore

Antes de seguir, crea un archivo `.gitignore` con al menos estas líneas: `node_modules`, `dist`, `.env`. Esto evita subir a tu repositorio Git la carpeta de dependencias (pesada y reinstalable), el código compilado (se regenera) y tus secretos (variables de entorno). Es una práctica imprescindible.

★ TIPS DEL EXPERTO

- ✦ Los scripts de `package.json` se ejecutan con `npm run <nombre>` (ej. `npm run dev`). Las excepciones son `start` y `test`, que funcionan también sin `run`.
- ✦ El `^` antes de la versión (`^4.19.2`) significa "compatible con": npm puede instalar parches y versiones menores, pero no cambios mayores que rompan compatibilidad.
- ✦ **Nunca subas `node_modules` a Git.** Puede pesar cientos de MB y se reconstruye con `npm install`. Por eso va en `.gitignore`.
- ✦ `npx` ejecuta paquetes sin instalarlos globalmente. `npx tsc --init` corre el TypeScript del proyecto, garantizando que uses la versión correcta.



PRACTICA LO APRENDIDO



RETO 1

Crea el proyecto

Sigue los pasos de la terminal para crear tu proyecto `api-tareas`. Abre el `package.json` generado y agrega manualmente los scripts `dev`, `build` y `start`. Crea también el `.gitignore`. Verifica que `node_modules`, `package.json` y `tsconfig.json` existan en tu carpeta.



RETO 2

Explora las dependencias

Ejecuta `npm list --depth=0` para ver tus dependencias directas. Luego abre la carpeta `node_modules` y observa cuántos paquetes hay realmente (muchos más de los que instalaste: son las dependencias de tus dependencias). Investiga qué hace `ts-node-dev` y por qué lo usamos solo en desarrollo.



LECCIÓN 07

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

TU PRIMER SERVIDOR EXPRESS

Levantar un servidor web con Express y TypeScript

CONCEPTO

Express es el framework web más popular de Node.js: simplifica enormemente la creación de servidores HTTP y APIs. Con unas pocas líneas defines rutas (qué responder según la URL y el método) y levantas un servidor que escucha peticiones. En esta lección crearás tu primer servidor Express en TypeScript, lo ejecutarás con recarga automática y verás tu primera respuesta en el navegador.



EXPLICACIÓN DETALLADA

Crear un servidor HTTP con Node puro es posible, pero tedioso: tendrías que parsear URLs a mano, manejar métodos, leer cuerpos de peticiones byte a byte. **Express** abstrae toda esa complejidad. Su API gira en torno a un objeto `app` que creas con `express()`. Sobre ese objeto registras **rutas** con métodos como `app.get(ruta, manejador)` y `app.post(...)`, y finalmente llamas a `app.listen(puerto)` para empezar a escuchar peticiones en ese puerto.

Cada **manejador de ruta** es una función que recibe dos objetos clave: `req` (request, la petición entrante con sus datos: parámetros, cuerpo, headers) y `res` (response, con métodos para construir la respuesta: `res.json(...)`, `res.status(...)`, `res.send(...)`). Con TypeScript, estos objetos están tipados gracias a `@types/express`, así que el editor te autocompletará todos sus métodos y te avisará si los usas mal.

Para que Express entienda cuerpos de peticiones en formato JSON (que necesitaremos para POST y PUT), debemos activar el **middleware** `express.json()` con `app.use(express.json())`. Un middleware es una función que se ejecuta antes de tus manejadores y puede transformar la petición; `express.json()` lee el cuerpo JSON entrante y lo deja disponible en `req.body`. Veremos los middlewares en profundidad en la lección 10.

Durante el desarrollo usaremos `ts-node-dev`, que ejecuta TypeScript directamente (sin compilar manualmente) y **reinicia el servidor automáticamente** cada vez que guardas un cambio. Esto acelera muchísimo el desarrollo: editas, guardas, y el servidor ya está actualizado. El script `dev` de nuestro `package.json` lo invoca. En producción, en cambio, compilaremos a JavaScript y ejecutaremos con `node` directamente (lección 19).

EJEMPLO PRÁCTICO

 **Caso:** Crear el servidor más simple posible: responde un saludo y devuelve datos en JSON.

 **src/server.ts**

```
import express, { Request, Response } from 'express';

// Crear la aplicación Express
const app = express();
const PORT = 3000;

// Middleware: permite leer JSON en el cuerpo de las peticiones
app.use(express.json());

// Ruta raíz: responde con texto plano
app.get('/', (req: Request, res: Response) => {
  res.send('¡Mi primer servidor Express con TypeScript!');
});

// Ruta que devuelve JSON
app.get('/api/saludo', (req: Request, res: Response) => {
  res.json({
    mensaje: 'Hola desde la API',
    fecha: new Date().toISOString(),
  });
});

// Ruta con parámetro dinámico en la URL
app.get('/api/saludo/:nombre', (req: Request, res: Response) => {
  const { nombre } = req.params;
  res.json({ mensaje: `Hola, ${nombre}!` });
});

// Iniciar el servidor
app.listen(PORT, () => {
  console.log(`🚀 Servidor corriendo en http://localhost:${PORT}`);
});
```

EJEMPLO PRÁCTICO

 **Caso:** Ejecutar el servidor en modo desarrollo y probarlo.

TERMINAL

```
# Ejecuta el servidor en modo desarrollo (con recarga automática)
$ npm run dev

🚀 Servidor corriendo en http://localhost:3000

# Abre tu navegador en http://localhost:3000
```

```
# o prueba la API desde otra terminal con curl:

$ curl http://localhost:3000/api/saludo
{"mensaje":"Hola desde la API","fecha":"2026-06-02T15:30:00.000Z"}

$ curl http://localhost:3000/api/saludo/Maria
{"mensaje":"Hola, Maria!"}
```

✅ ¡LO LOGRASTE!

Si ves el mensaje "🚀 Servidor corriendo..." y obtienes respuestas en el navegador o con curl, ya tienes un servidor backend funcionando. Todo lo demás en este tutorial es construir sobre esta base: más rutas, más lógica, validación, base de datos y despliegue. El esqueleto ya está vivo.

★ TIPS DEL EXPERTO

- ✦ El **puerto 3000** es una convención para desarrollo en Node. Puedes usar otro, pero asegúrate de que no esté ocupado por otra aplicación.
- ✦ Los **parámetros de ruta** (:nombre) capturan partes dinámicas de la URL y llegan en `req.params`. Son la base para rutas como `/tareas/:id`.
- ✦ Usa `res.json(objeto)` para responder JSON: pone automáticamente el header `Content-Type: application/json` y serializa el objeto. No uses `res.send` para objetos.
- ✦ Si cambias el código y el servidor no se actualiza, revisa que lo estés corriendo con `npm run dev` (que usa ts-node-dev) y no con `node`.

🚀 PRACTICA LO APRENDIDO



RETO 1

Agrega rutas nuevas

Sobre el servidor de la lección, agrega dos rutas GET: `/api/hora` que devuelva la hora actual en JSON, y `/api/suma/:a/:b` que reciba dos números en la URL y devuelva su suma. Recuerda que los parámetros llegan como texto: tendrás que convertirlos con `Number(...)`. Pruébalas con curl.



RETO 2

Maneja el puerto ocupado

Investiga qué pasa si intentas levantar dos servidores en el mismo puerto 3000 (verás el error [EADDRINUSE](#)). Modifica tu código para que el puerto se lea de una variable, con valor por defecto 3000. Esto nos preparará para la lección de variables de entorno.



LECCIÓN 08

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

ESTRUCTURA DE CARPETAS PROFESIONAL

Organizar el código en capas para que escale y se mantenga

CONCEPTO

A medida que una API crece, meter todo en un solo archivo se vuelve inmanejable. La solución profesional es separar el código en **capas** con responsabilidades claras: las **rut**as definen las URLs, los **controladores** manejan las peticiones y respuestas, los **servicios** contienen la lógica de negocio, y los **modelos/tipos** definen la forma de los datos. Esta separación (inspirada en la arquitectura por capas) hace el código más fácil de entender, probar y modificar.



EXPLICACIÓN DETALLADA

La idea central es la **separación de responsabilidades**: cada archivo y cada carpeta tiene un único trabajo bien definido. Cuando todo está mezclado, un cambio pequeño puede romper cosas inesperadas y encontrar un bug es buscar una aguja en un pajar. Cuando cada capa está aislada, sabes exactamente dónde buscar: ¿problema con una URL? mira las rutas; ¿lógica de negocio mal? mira los servicios; ¿datos mal formados? mira los modelos.

El **flujo de una petición** atraviesa las capas en orden. Llega una petición HTTP → el archivo de **rut**as la dirige al **controlador** correcto → el controlador extrae los datos de **req**, llama al **servicio** apropiado y construye la respuesta → el **servicio** ejecuta la lógica de negocio (validar reglas, calcular, coordinar) y usa el **modelo/repositorio** para leer o escribir datos → el resultado vuelve hacia arriba hasta convertirse en la respuesta HTTP. Cada capa solo conoce a la siguiente, no a todas.

Esta arquitectura hace el código **testable**. Como la lógica de negocio vive en los servicios (funciones puras que reciben datos y devuelven resultados, sin saber nada de HTTP), puedes probarla sin levantar un servidor: le pasas datos de entrada y verificas la salida. Los controladores, al ser delgados (solo traducen entre HTTP y servicios), tienen poca lógica que pueda fallar. Veremos cómo escribir estos tests en la lección 18.

No existe una única estructura "correcta": hay variantes (por capas técnicas, por funcionalidades, hexagonal, etc.). Para una API pequeña o mediana, la estructura por capas técnicas que usaremos —carpetas **routes**, **controllers**, **services**, **models**, **middlewares**— es clara, común y suficiente.

Lo importante no es la estructura exacta, sino mantener la **disciplina de separar responsabilidades** y ser consistente en todo el proyecto.



EJEMPLO PRÁCTICO



Caso: La estructura de carpetas que construiremos a lo largo del tutorial.



estructura-del-proyecto

```
api-tareas/  
  src/  
    server.ts      Todo el código fuente TypeScript  
    app.ts         Punto de entrada: arranca el servidor  
    config/  
      env.ts       Carga y valida variables de entorno  
    routes/  
      tareas.routes.ts Define las URLs de tareas  
    controllers/  
      tareas.controller.ts Maneja req/res de tareas  
    services/  
      tareas.service.ts Lógica de negocio de tareas  
    models/  
      tarea.model.ts  Tipos e interfaces de Tarea  
    middlewares/  
      errorHandler.ts Manejo centralizado de errores  
      validar.ts      Validación de datos de entrada  
    utils/  
      AppError.ts     Clase de error personalizada  
  tests/  
    tareas.test.ts    Pruebas automatizadas  
  dist/              JS compilado (generado, no se edita)  
  .env               Variables de entorno (NO subir a Git)  
  .gitignore  
  package.json  
  tsconfig.json
```



FLUJO DE UNA PETICIÓN POR LAS CAPAS



flujo-de-capas

Petición HTTP entrante

[routes] ¿Qué URL y método? dirige al controlador

```
[ controller ]   Lee req, llama al servicio, arma la respuesta

[ service ]      Lógica de negocio: validar, calcular, decidir

[ model/repo ]   Lee o escribe los datos (memoria, archivo, BD)
```

Respuesta HTTP (JSON + código de estado)

★ TIPS DEL EXPERTO

- ✦ Nombra los archivos por **responsabilidad y recurso**: `tareas.controller.ts`, `tareas.service.ts`. Al ver el nombre sabes exactamente qué hace.
- ✦ Los **controladores deben ser delgados**: solo traducen entre HTTP y la lógica. Si un controlador tiene reglas de negocio complejas, muévelas al servicio.
- ✦ Los **servicios no deben conocer HTTP**: no reciben `req/res`, sino datos planos. Así puedes reutilizarlos y probarlos sin un servidor.
- ✦ Separa `server.ts` (arranca el servidor) de `app.ts` (configura Express). Esto facilita los tests: puedes importar `app` sin levantar el puerto.

🚀 PRACTICA LO APRENDIDO



RETO 1

Crea la estructura

Crea todas las carpetas de la estructura mostrada dentro de `src/` (config, routes, controllers, services, models, middlewares, utils) y la carpeta `tests/`. Por ahora pueden estar vacías; las iremos llenando. Tener la estructura lista de antemano te dará claridad sobre dónde va cada cosa.



RETO 2

Separa app y server

Refactoriza el servidor de la lección 07: mueve la configuración de Express (crear `app`, los middlewares y las rutas) a un archivo `src/app.ts` que exporte `app`. Deja en `src/server.ts` solo la importación de `app` y el `app.listen(...)`. Verifica que todo siga funcionando con `npm run dev`.



LECCIÓN 09

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

RUTAS Y CONTROLADORES

Separar la definición de URLs de la lógica que las maneja

CONCEPTO

Una **ruta** asocia una URL y un método HTTP con una función que la maneja. Un **controlador** es esa función: recibe la petición, coordina el trabajo y envía la respuesta. En Express, agrupamos rutas relacionadas usando un **'Router'**, un mini-aplicación que maneja un conjunto de rutas (todas las de `/tareas`, por ejemplo) y luego se monta en la app principal. Separar rutas de controladores mantiene cada archivo enfocado y legible.



EXPLICACIÓN DETALLADA

El **'Router'** de Express te permite definir rutas en un archivo separado y luego montarlas en la app con un prefijo. Por ejemplo, defines un router con las rutas `/`, `/:id` para tareas, y lo montas con `app.use('/api/tareas', tareasRouter)`. Así, la ruta `/` del router se convierte en `/api/tareas`, y `/:id` en `/api/tareas/:id`. Esto evita repetir el prefijo en cada ruta y agrupa lógicamente los endpoints de cada recurso.

El archivo de **rutas** solo debe contener el mapeo URL → controlador, sin lógica. Se lee como un índice: "GET a la raíz va a `listar`, POST a la raíz va a `crear`, GET con id va a `obtenerPorId`". Esta claridad es valiosa: cualquiera que abra el archivo de rutas entiende de inmediato qué endpoints expone el recurso, sin perderse en detalles de implementación.

Los **controladores** son las funciones que efectivamente manejan cada petición. Reciben `req` y `res`, extraen los datos necesarios (parámetros de URL con `req.params`, cuerpo con `req.body`, query strings con `req.query`), llaman a la capa de servicio para hacer el trabajo real, y construyen la respuesta con el código de estado y los datos apropiados. Un buen controlador es corto: delega la lógica compleja al servicio y se concentra en la traducción HTTP.

En esta lección, los controladores aún tendrán datos "hardcodeados" (en memoria, dentro del propio controlador) para que te concentres en entender el flujo rutas → controlador → respuesta. En las lecciones 11 y 12 moveremos esos datos y la lógica a la capa de servicio, dejando los controladores delgados como deben ser. Es un proceso incremental: primero que funcione, luego que esté bien estructurado.

EJEMPLO PRÁCTICO

 **Caso:** Crear el router y el controlador de tareas, con datos temporales en memoria.

 **src/controllers/tareas.controller.ts**

```
import { Request, Response } from 'express';

// Datos temporales en memoria (luego irán al servicio)
let tareas = [
  { id: 1, titulo: 'Aprender Node.js', completada: true },
  { id: 2, titulo: 'Construir una API', completada: false },
];

// GET /api/tareas → Listar todas
export function listarTareas(req: Request, res: Response) {
  res.json(tareas);
}

// GET /api/tareas/:id → obtener una
export function obtenerTarea(req: Request, res: Response) {
  const id = Number(req.params.id);
  const tarea = tareas.find((t) => t.id === id);

  if (!tarea) {
    return res.status(404).json({ error: `Tarea ${id} no encontrada` });
  }
  res.json(tarea);
}

// POST /api/tareas → crear
export function crearTarea(req: Request, res: Response) {
  const { titulo } = req.body;
  const nueva = {
    id: tareas.length + 1,
    titulo,
    completada: false,
  };
  tareas.push(nueva);
  res.status(201).json(nueva);
}
```

 **src/routes/tareas.routes.ts**

```
import { Router } from 'express';
import {
  listarTareas,
  obtenerTarea,
  crearTarea,
} from '../controllers/tareas.controller';
```

```
const router = Router();

// Cada ruta apunta a su controlador (sin lógica aquí)
router.get('/', listarTareas);
router.get('/:id', obtenerTarea);
router.post('/', crearTarea);

export default router;
```

src/app.ts

```
import express from 'express';
import tareasRouter from './routes/tareas.routes';

const app = express();
app.use(express.json());

// Montar el router de tareas bajo el prefijo /api/tareas
app.use('/api/tareas', tareasRouter);

// Ruta de salud (health check)
app.get('/health', (req, res) => res.json({ estado: 'ok' }));

export default app;
```

TERMINAL

```
# Levanta el servidor y prueba los endpoints
$ npm run dev

$ curl http://localhost:3000/api/tareas
[{"id":1,"titulo":"Aprender Node.js","completada":true}, ...]

$ curl http://localhost:3000/api/tareas/2
{"id":2,"titulo":"Construir una API","completada":false}

# Crear una tarea (POST con cuerpo JSON)
$ curl -X POST http://localhost:3000/api/tareas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Desplegar a produccion"}'
{"id":3,"titulo":"Desplegar a produccion","completada":false}
```

★ TIPS DEL EXPERTO

- ✦ Usa `return res.status(...).json(...)` cuando quieras cortar la ejecución del controlador. Sin el `return`, el código seguiría y podría enviar dos respuestas (error).
- ✦ Recuerda que `req.params` siempre llega como **texto**. Convierte los IDs numéricos con `Number(req.params.id)` antes de compararlos.
- ✦ Un **Router** por recurso (`tareas.routes.ts`, `usuarios.routes.ts`) mantiene el código organizado. Móntalos todos en `app.ts` con sus prefijos.
- ✦ Añadir una ruta `/health` que devuelve `{ estado: 'ok' }` es una práctica estándar: los servicios de despliegue la usan para verificar que tu app está viva.



PRACTICA LO APRENDIDO



RETO 1

Completa el CRUD de rutas

Agrega al router y al controlador las rutas que faltan: `PUT /:id` (reemplazar), `PATCH /:id` (modificar parcial) y `DELETE /:id` (borrar). Implementa cada controlador trabajando sobre el array en memoria. Devuelve los códigos correctos: 200 para PUT/PATCH, 204 para DELETE, 404 si el id no existe.



RETO 2

Filtrado por query string

Modifica `listarTareas` para que acepte un query string opcional `?completada=true` o `?completada=false`. Si está presente, filtra las tareas por ese estado; si no, devuelve todas. Los query strings llegan en `req.query`. Pruébalo con `curl "http://localhost:3000/api/tareas?completada=true"`.



LECCIÓN 10

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

MIDDLEWARES

Funciones que se ejecutan entre la petición y la respuesta

CONCEPTO

Un **middleware** es una función que se ejecuta **durante el ciclo de una petición**, antes de que llegue al controlador final. Recibe `req`, `res` y una función `next` que pasa el control al siguiente middleware. Los middlewares sirven para tareas transversales: registrar peticiones (logging), parsear el cuerpo JSON, verificar autenticación, validar datos, manejar errores. Son el mecanismo de extensibilidad central de Express.



EXPLICACIÓN DETALLADA

Express procesa cada petición como una **cadena de middlewares**: la petición entra y pasa por cada middleware registrado, en orden, hasta llegar al controlador que envía la respuesta. Cada middleware puede hacer tres cosas: ejecutar código (loguear, modificar `req`), terminar la petición (enviar una respuesta y cortar la cadena), o llamar a `next()` para pasar al siguiente. Si un middleware no llama a `next()` ni envía respuesta, la petición se queda colgada para siempre.

El parámetro `next` es la clave del mecanismo. Llamar a `next()` sin argumentos pasa al siguiente middleware normal. Llamar a `next(error)` con un argumento salta directamente a los middlewares de **manejo de errores** (que tienen cuatro parámetros: `err`, `req`, `res`, `next`). Este patrón nos permitirá centralizar todo el manejo de errores en un solo lugar, como veremos en la lección 14.

Hay middlewares **integrados** (como `express.json()` que parsea cuerpos JSON), **de terceros** (como `cors` para permitir peticiones de otros dominios, o `morgan` para logging) y **propios** (los que escribes tú). Se registran con `app.use(middleware)` para que se apliquen a todas las rutas, o se pasan como argumentos a rutas específicas para aplicarlos solo allí. El **orden de registro importa muchísimo**: se ejecutan en el orden en que los declaras.

Los middlewares propios son simplemente funciones con la firma `(req, res, next)`. Por ejemplo, un middleware de logging registra cada petición que llega; uno de autenticación verifica que el usuario tenga un token válido antes de dejar pasar; uno de validación comprueba que el cuerpo tenga los campos requeridos. Esta capacidad de "interceptar" peticiones de forma reutilizable es lo que hace a Express tan flexible y potente para construir APIs robustas.

EJEMPLO PRÁCTICO

 **Caso:** Crear un middleware de logging propio y usar el middleware de terceros CORS.

 **src/middlewares/logger.ts**

```
import { Request, Response, NextFunction } from 'express';

// Middleware propio: registra cada petición entrante
export function logger(req: Request, res: Response, next: NextFunction) {
  const ahora = new Date().toISOString();
  console.log(`[${ahora}] ${req.method} ${req.url}`);
  next(); // ¡importante! pasa al siguiente middleware
}

// Middleware propio: mide el tiempo de respuesta
export function medirTiempo(req: Request, res: Response, next: NextFunction) {
  const inicio = Date.now();
  res.on('finish', () => {
    const ms = Date.now() - inicio;
    console.log(` ↳ ${res.statusCode} (${ms}ms)`);
  });
  next();
}
```

 **src/app.ts**

```
import express from 'express';
import cors from 'cors';
import tareasRouter from './routes/tareas.routes';
import { logger, medirTiempo } from './middlewares/logger';

const app = express();

// === Middlewares globales (el ORDEN importa) ===
app.use(cors()); // 1. permite peticiones de otros orígenes
app.use(express.json()); // 2. parsea cuerpos JSON → req.body
app.use(logger); // 3. registra cada petición
app.use(medirTiempo); // 4. mide tiempo de respuesta

// === Rutas ===
app.use('/api/tareas', tareasRouter);

export default app;
```

TERMINAL

```
# Instala CORS y sus tipos
```

```
$ npm install cors
$ npm install -D @types/cors

# Al hacer peticiones, verás los logs en la consola del servidor:
[2026-06-02T15:30:00.000Z] GET /api/tareas
  ↳ 200 (3ms)
[2026-06-02T15:30:05.000Z] POST /api/tareas
  ↳ 201 (5ms)
```

✓ ¿QUÉ ES CORS?

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad del navegador que bloquea, por defecto, las peticiones entre dominios distintos. Si tu frontend está en miapp.com y tu API en api.miapp.com, el navegador bloqueará las peticiones a menos que la API lo permita explícitamente. El middleware `cors()` agrega los headers necesarios para autorizarlas. En producción, configúralo para permitir solo los dominios de confianza.

★ TIPS DEL EXPERTO

- ✦ Siempre llama a `next()` en un middleware que no envía respuesta. Si lo olvidas, la petición se queda colgada y el cliente espera para siempre.
- ✦ El **orden de los `app.use(...)` importa**. Pon primero los middlewares globales (`cors`, `json`, `logging`) y al final el manejo de errores.
- ✦ Para aplicar un middleware solo a ciertas rutas, pásalo como argumento: `router.post('/', validar, crearTarea)`. Se ejecuta antes del controlador.
- ✦ Usa `morgan` (un middleware de logging de terceros muy popular) en proyectos reales en lugar de escribir tu propio logger. Es más completo y configurable.

🚀 PRACTICA LO APRENDIDO



RETO 1

Middleware de API Key

Crea un middleware `verificarApiKey` que revise si la petición trae un header `x-api-key` con el valor `'secreto123'`. Si lo trae, llama a `next()`. Si no, responde **401 Unauthorized** con un mensaje de error. Aplícalo solo a las rutas de tareas y pruébalo con y sin el header usando curl (`-H "x-api-key: secreto123"`).



RETO 2

Middleware 404

Crea un middleware que se ejecute **al final** de todas las rutas (después de montarlas) y que responda **404 Not Found** con `{ error: 'Ruta no encontrada' }` para cualquier URL que no coincida con ninguna ruta definida. Pruébalo pidiendo una URL inexistente como `/api/cualquiercosa`.



LECCIÓN 11

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

MODELO DE DATOS Y CAPA DE SERVICIO*Definir la forma de los datos y aislar la lógica de negocio***CONCEPTO**

El **modelo** define la **forma de los datos** que maneja tu API, usando interfaces y tipos de TypeScript. La **capa de servicio** contiene la **lógica de negocio**: las operaciones reales sobre esos datos (buscar, crear, actualizar, borrar), sin saber nada de HTTP. Al separar el servicio del controlador, la lógica se vuelve reutilizable y fácil de probar, y los controladores quedan delgados: solo traducen entre HTTP y el servicio.

**EXPLICACIÓN DETALLADA**

El **modelo** en TypeScript se define con **interface** o **type**. Una interface describe qué propiedades tiene un objeto y de qué tipo es cada una. Para nuestra API de tareas, definiremos una interface **Tarea** con **id** (número), **titulo** (texto), **completada** (booleano) y una fecha de creación. Tener este tipo centralizado significa que todo el código que maneje tareas usará exactamente la misma forma, y TypeScript te avisará si en algún lugar olvidas un campo o usas un tipo equivocado.

También es útil definir tipos auxiliares para las operaciones. Por ejemplo, cuando creas una tarea, el cliente envía solo el **titulo** (el **id**, **completada** y la fecha los pone el servidor). Para eso definimos un tipo **CrearTareaDTO** (Data Transfer Object) que describe exactamente qué espera recibir la operación de creación. Estos DTOs hacen explícito el contrato de cada operación y evitan confusiones sobre qué datos viajan en cada momento.

La **capa de servicio** es donde vive la lógica de negocio de verdad. Es un módulo con funciones como **obtenerTodas()**, **obtenerPorId(id)**, **crear(datos)**, **actualizar(id, datos)**, **eliminar(id)**. Crucialmente, estas funciones **no conocen HTTP**: no reciben **req** ni **res**, reciben y devuelven datos planos de TypeScript. Esto las hace independientes del framework: podrías cambiar de Express a otro framework sin tocar los servicios, y puedes probarlas llamándolas directamente con datos de prueba.

Por ahora, el servicio guardará las tareas en un **array en memoria**. Esto es perfecto para aprender y desarrollar: es simple y rápido. La limitación es que los datos se pierden al reiniciar el servidor. En la lección 16 reemplazaremos este almacenamiento en memoria por persistencia real (un archivo JSON o una base de datos), y gracias a la separación en capas, **solo tendremos que cambiar el**

servicio: los controladores y rutas no se enterarán del cambio. Esa es la magia de la arquitectura por capas.



EJEMPLO PRÁCTICO



Caso: Definir el modelo Tarea y la capa de servicio con almacenamiento en memoria.

 **src/models/tarea.model.ts**

```
// La forma de una Tarea completa (como se guarda y devuelve)
export interface Tarea {
  id: number;
  titulo: string;
  completada: boolean;
  creadaEn: string; // fecha ISO
}

// Datos que envía el cliente al CREAR (sin id ni fecha)
export interface CrearTareaDTO {
  titulo: string;
}

// Datos que envía el cliente al ACTUALIZAR (todos opcionales)
export interface ActualizarTareaDTO {
  titulo?: string;
  completada?: boolean;
}
```

 **src/services/tareas.service.ts**

```
import { Tarea, CrearTareaDTO, ActualizarTareaDTO } from '../models/tarea.model';

// Almacenamiento en memoria (temporal, hasta la Lección 16)
let tareas: Tarea[] = [
  { id: 1, titulo: 'Aprender Node.js', completada: true, creadaEn: new Date().toISOString() },
  { id: 2, titulo: 'Construir una API', completada: false, creadaEn: new Date().toISOString() },
];
let siguienteId = 3;

// Obtener todas las tareas
export function obtenerTodas(): Tarea[] {
  return tareas;
}

// Obtener una tarea por id (undefined si no existe)
```

```
export function obtenerPorId(id: number): Tarea | undefined {
  return tareas.find((t) => t.id === id);
}

// Crear una tarea nueva
export function crear(datos: CrearTareaDTO): Tarea {
  const nueva: Tarea = {
    id: siguienteId++,
    titulo: datos.titulo,
    completada: false,
    creadaEn: new Date().toISOString(),
  };
  tareas.push(nueva);
  return nueva;
}

// Actualizar una tarea existente (parcial)
export function actualizar(id: number, datos: ActualizarTareaDTO): Tarea | undefined {
  const tarea = obtenerPorId(id);
  if (!tarea) return undefined;

  if (datos.titulo !== undefined) tarea.titulo = datos.titulo;
  if (datos.completada !== undefined) tarea.completada = datos.completada;
  return tarea;
}

// Eliminar una tarea (devuelve true si existía)
export function eliminar(id: number): boolean {
  const cantidadAntes = tareas.length;
  tareas = tareas.filter((t) => t.id !== id);
  return tareas.length < cantidadAntes;
}
```

✓ EL SERVICIO NO SABE DE HTTP

Fíjate que ninguna función del servicio recibe `req` o `res`, ni devuelve códigos de estado. Reciben números y objetos, y devuelven tareas o booleanos. Esa independencia es deliberada: el servicio solo se ocupa de la lógica de las tareas. El controlador será quien traduzca estos resultados a respuestas HTTP en la próxima lección.

★ TIPS DEL EXPERTO

- ✦ Usa `interface` para describir la forma de tus datos. TypeScript verificará que todo objeto `Tarea` tenga exactamente esos campos con esos tipos.

- ✦ El `? en titulo?: string` marca el campo como **opcional**. Es ideal para DTOs de actualización parcial (PATCH), donde el cliente envía solo lo que quiere cambiar.
- ✦ El tipo de retorno `Tarea | undefined` documenta que la búsqueda **puede no encontrar nada**. TypeScript te obligará a manejar ese caso, previniendo errores de `null`.
- ✦ Los **DTOs** (`CrearTareaDTO`, `ActualizarTareaDTO`) hacen explícito qué datos viajan en cada operación. Son documentación viva y protección de tipos a la vez.

PRACTICA LO APRENDIDO



RETO 1

Añade búsqueda y conteo

Agrega al servicio dos funciones: `buscarPorTitulo(texto: string): Tarea[]` que devuelva las tareas cuyo título contenga ese texto (sin importar mayúsculas), y `contarPendientes(): number` que devuelva cuántas tareas no están completadas. Recuerda: sin tocar HTTP, solo lógica.



RETO 2

Modela un nuevo recurso

Diseña el modelo (interface) y la capa de servicio en memoria para un recurso `Usuario` con `id`, `nombre`, `email` y `activo`. Define los DTOs de creación y actualización. Implementa las cinco operaciones CRUD del servicio. No necesitas conectarlo a rutas todavía: el objetivo es practicar la separación modelo/servicio.



LECCIÓN 12

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

CRUD COMPLETO

Conectar controladores delgados con la capa de servicio



CONCEPTO

CRUD son las cuatro operaciones básicas sobre datos: **Create** (crear), **Read** (leer), **Update** (actualizar) y **Delete** (borrar). En esta lección conectamos todo lo construido: los controladores ahora delegan en la capa de servicio (creada en la lección 11) y se vuelven **delgados**, ocupándose solo de leer la petición, llamar al servicio y construir la respuesta HTTP con el código de estado correcto. El resultado es una API REST completa y bien estructurada.



EXPLICACIÓN DETALLADA

Ahora que la lógica vive en el servicio, los controladores se simplifican enormemente. Cada controlador sigue el mismo patrón: extrae los datos de la petición (`req.params`, `req.body`), llama a la función del servicio correspondiente, y según el resultado, responde con el código y los datos apropiados. Comparado con la lección 9, donde los controladores tenían el array y la lógica dentro, ahora son una capa delgada de traducción HTTP ↔ servicio.

El mapeo de operaciones CRUD a métodos HTTP y códigos de estado es directo: **Create** es **POST** y responde **201 Created** con la tarea creada. **Read** es **GET** y responde **200 OK** con los datos (o **404** si se pide algo que no existe). **Update** es **PUT/PATCH** y responde **200 OK** con la tarea actualizada (o **404**). **Delete** es **DELETE** y responde **204 No Content** (o **404**). Aplicar estos códigos correctamente hace que tu API sea predecible y profesional.

Un detalle importante es **manejar el caso "no encontrado"** en cada operación que recibe un id. El servicio devuelve `undefined` (o `false` en el delete) cuando el id no existe; el controlador detecta eso y responde **404** con un mensaje claro. Olvidar este chequeo es un error común que lleva a respuestas confusas o caídas del servidor. TypeScript ayuda aquí: como el servicio devuelve `Tarea | undefined`, el compilador te recuerda manejar el caso `undefined`.

Con esta lección tienes una **API REST CRUD completa y funcional**. Puedes listar tareas, obtener una específica, crear nuevas, actualizarlas (total o parcialmente) y borrarlas, todo con los métodos HTTP y códigos de estado correctos, y con el código bien organizado en capas. Las siguientes lecciones la harán más robusta: validación de datos (13), manejo centralizado de errores (14), configuración (15) y persistencia real (16). Pero la funcionalidad central ya está aquí.

EJEMPLO PRÁCTICO

 **Caso:** El controlador completo de tareas, ahora delgado y delegando en el servicio.

 **src/controllers/tareas.controller.ts**

```
import { Request, Response } from 'express';
import * as tareasService from '../services/tareas.service';

// GET /api/tareas → READ (todas)
export function listarTareas(req: Request, res: Response) {
  const tareas = tareasService.obtenerTodas();
  res.status(200).json(tareas);
}

// GET /api/tareas/:id → READ (una)
export function obtenerTarea(req: Request, res: Response) {
  const id = Number(req.params.id);
  const tarea = tareasService.obtenerPorId(id);

  if (!tarea) {
    return res.status(404).json({ error: `Tarea ${id} no encontrada` });
  }
  res.status(200).json(tarea);
}

// POST /api/tareas → CREATE
export function crearTarea(req: Request, res: Response) {
  const { titulo } = req.body;
  const nueva = tareasService.crear({ titulo });
  res.status(201).json(nueva);
}

// PUT/PATCH /api/tareas/:id → UPDATE
export function actualizarTarea(req: Request, res: Response) {
  const id = Number(req.params.id);
  const { titulo, completada } = req.body;
  const actualizada = tareasService.actualizar(id, { titulo, completada });

  if (!actualizada) {
    return res.status(404).json({ error: `Tarea ${id} no encontrada` });
  }
  res.status(200).json(actualizada);
}

// DELETE /api/tareas/:id → DELETE
export function eliminarTarea(req: Request, res: Response) {
  const id = Number(req.params.id);
  const eliminada = tareasService.eliminar(id);

  if (!eliminada) {
    return res.status(404).json({ error: `Tarea ${id} no encontrada` });
  }
  res.status(204).send(); // 204: sin cuerpo
}
```

```
}
```

src/routes/tareas.routes.ts

```
import { Router } from 'express';
import {
  listarTareas, obtenerTarea, crearTarea,
  actualizarTarea, eliminarTarea,
} from '../controllers/tareas.controller';

const router = Router();

router.get('/', listarTareas);           // READ todas
router.get('/:id', obtenerTarea);       // READ una
router.post('/', crearTarea);           // CREATE
router.put('/:id', actualizarTarea);    // UPDATE total
router.patch('/:id', actualizarTarea); // UPDATE parcial
router.delete('/:id', eliminarTarea);  // DELETE

export default router;
```

TERMINAL

```
# Prueba el CRUD completo con curl

# CREATE
$ curl -X POST localhost:3000/api/tareas \
  -H "Content-Type: application/json" -d '{"titulo":"Estudiar"}'
{"id":3,"titulo":"Estudiar","completada":false,"creadaEn":"..."}

# READ (una)
$ curl localhost:3000/api/tareas/3
{"id":3,"titulo":"Estudiar","completada":false,...}

# UPDATE parcial (marcar completada)
$ curl -X PATCH localhost:3000/api/tareas/3 \
  -H "Content-Type: application/json" -d '{"completada":true}'
{"id":3,"titulo":"Estudiar","completada":true,...}

# DELETE
$ curl -X DELETE localhost:3000/api/tareas/3 -i
HTTP/1.1 204 No Content
```

★ TIPS DEL EXPERTO

- ✦ Reutiliza el mismo controlador `actualizarTarea` para PUT y PATCH: como el servicio actualiza solo los campos presentes, funciona para ambos casos sin duplicar código.
- ✦ En `DELETE`, usa `res.status(204).send()` sin pasar datos. El código 204 significa "éxito sin contenido", así que no debe llevar cuerpo.
- ✦ Maneja **siempre** el caso `404` en operaciones con id. Es el error más común y el que más confunde a quien consume tu API si lo manejas mal.
- ✦ Fíjate lo **cortos** que quedaron los controladores al delegar en el servicio. Si un controlador crece mucho, probablemente tiene lógica que debería estar en el servicio.



PRACTICA LO APRENDIDO



RETO 1

Endpoint de estadísticas

Agrega un endpoint `GET /api/tareas/stats` que devuelva un objeto con: total de tareas, cuántas completadas y cuántas pendientes. Crea la función en el servicio y el controlador correspondiente. Cuidado con el orden de las rutas: `/stats` debe declararse antes de `/:id`, o Express interpretará "stats" como un id.



RETO 2

CRUD de otro recurso

Usando el modelo `Usuario` que creaste en el reto de la lección 11, implementa su CRUD completo: el controlador delgado con las cinco operaciones, las rutas montadas bajo `/api/usuarios`, y los códigos de estado correctos. Pruébalo entero con curl. Habrás construido tu segundo recurso REST por tu cuenta.



LECCIÓN 13

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

VALIDACIÓN DE DATOS

Nunca confíes en los datos que envía el cliente

CONCEPTO

La **validación** consiste en verificar que los datos que llegan en una petición cumplan las reglas esperadas **antes** de procesarlos: que los campos obligatorios estén presentes, que tengan el tipo correcto, que respeten longitudes y formatos. Es una regla de oro del backend: **nunca confíes en el cliente**. Validar previene datos corruptos, errores difíciles de rastrear y, sobre todo, vulnerabilidades de seguridad. Cuando los datos son inválidos, respondes **400 Bad Request** con un mensaje claro.



EXPLICACIÓN DETALLADA

El cliente puede enviar cualquier cosa: campos faltantes, tipos equivocados (un número donde esperabas texto), valores absurdos (una edad negativa), o incluso datos maliciosos diseñados para romper tu sistema. Si procesas estos datos sin validar, en el mejor caso guardas basura en tu base de datos, y en el peor abres la puerta a ataques. Por eso, la validación es la **primera línea de defensa** de tu API y debe ocurrir antes de cualquier lógica de negocio.

Puedes validar de dos formas: **manualmente**, escribiendo comprobaciones con `if` (rápido para casos simples), o con una **librería de validación** como `zod`, `joi` o `express-validator` (recomendado para proyectos reales, porque es declarativo, reutilizable y maneja casos complejos). En este tutorial mostraremos primero validación manual para que entiendas el concepto, y luego cómo encapsularla en un middleware reutilizable.

La validación encaja perfectamente como un **middleware** (lección 10): una función que se ejecuta antes del controlador, revisa `req.body`, y o bien llama a `next()` si los datos son válidos, o responde **400** y corta la cadena si no lo son. Esto mantiene los controladores limpios (solo reciben datos ya validados) y permite reutilizar la misma validación en varias rutas. Es el patrón profesional para validar entradas.

Una buena validación devuelve **mensajes de error claros y específicos**: no basta con "datos inválidos", sino "el campo título es obligatorio" o "completada debe ser un booleano". Idealmente, recopila **todos** los errores de una petición y los devuelve juntos, en lugar de uno por uno, para que el cliente pueda corregir todo de una vez. Estos mensajes son la diferencia entre una API frustrante y una que es un placer integrar.

EJEMPLO PRÁCTICO


Caso: Crear un middleware de validación reutilizable para crear y actualizar tareas.


src/middlewares/validarTarea.ts

```

import { Request, Response, NextFunction } from 'express';

// Valida los datos al CREAR una tarea
export function validarCrearTarea(
  req: Request, res: Response, next: NextFunction
) {
  const errores: string[] = [];
  const { titulo } = req.body;

  // El título es obligatorio
  if (titulo === undefined || titulo === null) {
    errores.push('El campo "titulo" es obligatorio');
  } else if (typeof titulo !== 'string') {
    errores.push('El campo "titulo" debe ser texto');
  } else if (titulo.trim().length === 0) {
    errores.push('El campo "titulo" no puede estar vacío');
  } else if (titulo.length > 100) {
    errores.push('El "titulo" no puede superar 100 caracteres');
  }

  // Si hay errores, responde 400 con todos juntos
  if (errores.length > 0) {
    return res.status(400).json({ errores });
  }
  next(); // datos válidos: continúa al controlador
}

// Valida los datos al ACTUALIZAR (campos opcionales)
export function validarActualizarTarea(
  req: Request, res: Response, next: NextFunction
) {
  const errores: string[] = [];
  const { titulo, completada } = req.body;

  if (titulo !== undefined) {
    if (typeof titulo !== 'string' || titulo.trim().length === 0) {
      errores.push('Si envías "titulo", debe ser texto no vacío');
    }
  }

  if (completada !== undefined && typeof completada !== 'boolean') {
    errores.push('El campo "completada" debe ser true o false');
  }

  if (errores.length > 0) {
    return res.status(400).json({ errores });
  }
}
    
```

```
    next();  
  }  
}
```

src/routes/tareas.routes.ts

```
import { Router } from 'express';  
import * as ctrl from '../controllers/tareas.controller';  
import {  
  validarCrearTarea,  
  validarActualizarTarea,  
} from '../middlewares/validarTarea';  
  
const router = Router();  
  
router.get('/', ctrl.listarTareas);  
router.get('/:id', ctrl.obtenerTarea);  
  
// El middleware de validación se ejecuta ANTES del controlador  
router.post('/', validarCrearTarea, ctrl.crearTarea);  
router.put('/:id', validarActualizarTarea, ctrl.actualizarTarea);  
router.patch('/:id', validarActualizarTarea, ctrl.actualizarTarea);  
router.delete('/:id', ctrl.eliminarTarea);  
  
export default router;
```

TERMINAL

```
# Intentar crear una tarea sin título → 400  
$ curl -X POST localhost:3000/api/tareas \  
  -H "Content-Type: application/json" -d '{}' -i  
HTTP/1.1 400 Bad Request  
{  
  "errores": ["El campo \"titulo\" es obligatorio"]  
}  
  
# Título vacío → 400  
$ curl -X POST localhost:3000/api/tareas \  
  -H "Content-Type: application/json" -d '{"titulo": ""}' -i  
HTTP/1.1 400 Bad Request  
{  
  "errores": ["El campo \"titulo\" no puede estar vacío"]  
}  
  
# Título válido → 201 Created  
$ curl -X POST localhost:3000/api/tareas \  
  -H "Content-Type: application/json" -d '{"titulo": "Válida"}'  
{  
  "id": 3, "titulo": "Válida", "completada": false, ...  
}
```

✓ EN PROYECTOS REALES: USA ZOD

La validación manual es excelente para aprender, pero en proyectos serios usa una librería como **Zod**. Defines un "esquema" (`z.object({ titulo: z.string().min(1).max(100) })`) y la librería valida automáticamente, genera mensajes y hasta infiere los tipos de TypeScript. Menos código, menos errores. El reto 2 te invita a explorarla.

★ TIPS DEL EXPERTO

- ✦ **Valida en un middleware**, no dentro del controlador. Así el controlador recibe datos ya limpios y la validación se reutiliza en varias rutas.
- ✦ Recopila **todos los errores** y devuélvelos juntos en un array. Es mucho mejor que el cliente vea de una vez todo lo que debe corregir.
- ✦ Valida **tipo, presencia y formato**: que el campo exista, que sea del tipo correcto, y que cumpla reglas (longitud, rango, patrón). Los tres importan.
- ✦ Para validaciones complejas (emails, fechas, objetos anidados), usa **Zod** o **express-validator**. Reinventar la validación a mano no escala.

🚀 PRACTICA LO APRENDIDO



RETO 1

Validación con reglas extra

Amplía `validarCrearTarea` para aceptar un campo opcional `prioridad` que, si viene, debe ser exactamente uno de estos valores: `'alta'`, `'media'` o `'baja'`. Si viene cualquier otro valor, agrega un error claro al array. Prueba con prioridades válidas e inválidas.



RETO 2

Explora Zod

Instala Zod (`npm install zod`). Reescribe la validación de creación de tareas usando un esquema de Zod: `const esquema = z.object({ titulo: z.string().min(1).max(100) })`. Usa `esquema.safeParse(req.body)` en el middleware y devuelve los errores que Zod genera. Compara cuánto código te ahorras frente a la versión manual.



LECCIÓN 14

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

MANEJO DE ERRORES CENTRALIZADO

Un solo lugar para capturar y responder todos los errores CONCEPTO

El **manejo centralizado de errores** consiste en capturar todos los errores de la aplicación en un único **middleware de errores** que decide cómo responder. En lugar de repetir bloques `try/catch` con respuestas de error en cada controlador, lanzas un error (o lo pasas con `next(error)`) y un middleware especial —que Express identifica por tener cuatro parámetros (`err`, `req`, `res`, `next`)— lo recibe, registra y responde con el código y mensaje adecuados. Esto mantiene los controladores limpios y la lógica de errores consistente.

 EXPLICACIÓN DETALLADA

Sin manejo centralizado, cada controlador tendría su propio `try/catch` con su propia forma de responder errores. El resultado: código repetido, respuestas inconsistentes (un endpoint devuelve `{ error }`, otro `{ message }`), y el riesgo de que un error no capturado tumbe el servidor. La solución es un **middleware de manejo de errores** al final de la cadena: todos los errores fluyen hacia él, y él decide la respuesta de forma uniforme.

Express reconoce un middleware como "de errores" porque tiene **cuatro parámetros**: (`err`, `req`, `res`, `next`). Cuando en cualquier parte llamas a `next(error)` con un argumento, o cuando ocurre una excepción en un controlador asíncrono que propagas, Express salta directo a este middleware, ignorando los middlewares y rutas normales restantes. Allí inspeccionas el error y construyes la respuesta. Debe registrarse **al final**, después de todas las rutas.

Para errores controlados (de negocio), creamos una **clase de error personalizada** `AppError` que extiende `Error` y añade un `statusCode`. Así, cuando en un servicio detectas que una tarea no existe, lanzas `throw new AppError('Tarea no encontrada', 404)`, y el middleware de errores lee ese `statusCode` para responder correctamente. Distingues entre errores operacionales esperados (que tienen su código) y errores inesperados (bugs), que se tratan como `500`.

Un detalle técnico: en Express 4, los errores que ocurren dentro de funciones `async` no se capturan automáticamente; debes envolverlos. Por eso usamos un pequeño helper `asyncHandler` que envuelve los controladores asíncronos y captura cualquier promesa rechazada, pasándola a `next()`. Con esto, puedes escribir controladores `async` con `await` sin preocuparte por los

try/catch: los errores fluyen solos al middleware central. (Express 5, más reciente, maneja esto de forma nativa.)



EJEMPLO PRÁCTICO



Caso: Crear la clase `AppError`, el helper `asyncHandler` y el middleware central de errores.

 `src/utils/AppError.ts`

```
// Error personalizado con código de estado HTTP
export class AppError extends Error {
  statusCode: number;
  esOperacional: boolean;

  constructor(mensaje: string, statusCode: number) {
    super(mensaje);
    this.statusCode = statusCode;
    this.esOperacional = true; // error esperado, no un bug
    Object.setPrototypeOf(this, AppError.prototype);
  }
}
```

 `src/utils/asyncHandler.ts`

```
import { Request, Response, NextFunction } from 'express';

// Envuelve controladores async y captura sus errores
type Controlador = (
  req: Request, res: Response, next: NextFunction
) => Promise<unknown>;

export function asyncHandler(fn: Controlador) {
  return (req: Request, res: Response, next: NextFunction) => {
    fn(req, res, next).catch(next); // cualquier error → next(error)
  };
}
```

 `src/middlewares/errorHandler.ts`

```
import { Request, Response, NextFunction } from 'express';
import { AppError } from '../utils/AppError';

// Middleware de manejo de errores (4 parámetros = Express lo reconoce)
export function errorHandler(
```

```
err: Error,  
req: Request,  
res: Response,  
next: NextFunction  
) {  
  // Error controlado (de negocio): usa su código  
  if (err instanceof AppError) {  
    return res.status(err.statusCode).json({ error: err.message });  
  }  
  
  // Error inesperado (bug): regístralo y responde 500 genérico  
  console.error('❗ Error inesperado:', err);  
  res.status(500).json({ error: 'Error interno del servidor' });  
}
```

src/controllers/tareas.controller.ts (fragmento)

```
import { Request, Response } from 'express';  
import * as tareasService from '../services/tareas.service';  
import { AppError } from '../utils/AppError';  
import { asyncHandler } from '../utils/asyncHandler';  
  
// Controlador que LANZA errores en vez de manejarlos aquí  
export const obtenerTarea = asyncHandler(async (req: Request, res: Response) => {  
  const id = Number(req.params.id);  
  const tarea = tareasService.obtenerPorId(id);  
  
  // En vez de responder 404 aquí, lanzamos el error  
  if (!tarea) {  
    throw new AppError(`Tarea ${id} no encontrada`, 404);  
  }  
  res.status(200).json(tarea);  
});
```

src/app.ts

```
import express from 'express';  
import tareasRouter from '../routes/tareas.routes';  
import { errorHandler } from '../middlewares/errorHandler';  
  
const app = express();  
app.use(express.json());  
  
app.use('/api/tareas', tareasRouter);  
  
// Ruta no encontrada (404 para URLs desconocidas)  
app.use((req, res) => {  
  res.status(404).json({ error: 'Ruta no encontrada' });  
});
```

```
});  
  
// ⚠ EL manejador de errores va SIEMPRE al final  
app.use(errorHandler);  
  
export default app;
```

⚠ EL ORDEN ES CRÍTICO

El middleware `errorHandler` debe registrarse con `app.use(...)` **después de todas las rutas**, al final de `app.ts`. Express ejecuta los middlewares en orden, y los de error solo se activan cuando algo llama a `next(error)`. Si lo pones antes de las rutas, nunca capturará nada.

★ TIPS DEL EXPERTO

- ✦ Lanza `throw new AppError(mensaje, codigo)` desde controladores y servicios. El middleware central se encarga del resto: respuesta uniforme y consistente.
- ✦ Distingue **errores operacionales** (esperados: 404, 400, 409) de **bugs** (inesperados: 500). Los primeros informan al cliente; los segundos se registran y se ocultan.
- ✦ **Nunca expongas stack traces al cliente** en producción: son información sensible. Regístralos del lado del servidor con `console.error` o un logger, y responde un mensaje genérico.
- ✦ El helper `asyncHandler` te libera de escribir `try/catch` en cada controlador async. Envuelve y olvídate: los errores fluyen solos al manejador central.

🚀 PRACTICA LO APRENDIDO



RETO 1

Refactoriza todo el CRUD

Convierte todos los controladores de tareas al patrón con `asyncHandler` y `AppError`. Elimina los `res.status(404)` repetidos y reemplázalos por `throw new AppError(...)`. Verifica que las respuestas de error sigan siendo correctas. Notarás cuánto se simplifican los controladores.



RETO 2

Error de conflicto

Imagina que las tareas no pueden tener títulos duplicados. En el servicio `crear`, verifica si ya existe una tarea con ese título y, si es así, lanza `new AppError('Ya`

`existe una tarea con ese título', 409)`. Comprueba que el middleware responda correctamente con el código 409 y el mensaje. Repasa por qué 409 es el código adecuado aquí.



LECCIÓN 15

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

VARIABLES DE ENTORNO Y CONFIGURACIÓN

Separar la configuración y los secretos del código

CONCEPTO

Las **variables de entorno** son valores de configuración que viven **fuera del código**, en el entorno donde se ejecuta la aplicación. Sirven para datos que cambian según el entorno (puerto, URL de la base de datos) y, sobre todo, para **secretos** (contraseñas, claves de API, tokens) que nunca deben escribirse en el código ni subirse a Git. En Node.js se acceden con `process.env`, y en desarrollo se cargan desde un archivo `.env` usando la librería `dotenv`.



EXPLICACIÓN DETALLADA

Escribir valores de configuración directamente en el código ("hardcodear") es un mal hábito por dos razones. Primera: la configuración cambia entre entornos. En tu máquina la base de datos está en `localhost`, pero en producción está en otro servidor. Si está hardcodeada, tendrías que cambiar el código para cada entorno. Segunda y más grave: los **secretos**. Si pones una contraseña o clave de API en el código y lo subes a un repositorio (incluso privado), queda expuesta para siempre en el historial de Git.

La solución es el principio de **configuración por entorno**: el código lee sus valores de `process.env`, y cada entorno provee sus propios valores. En desarrollo, usas un archivo `.env` (que va en `.gitignore`, nunca se sube) con líneas `CLAVE=valor`. La librería `dotenv` lee ese archivo al arrancar y carga sus valores en `process.env`. En producción, el proveedor de hosting (Render, Railway, etc.) te permite definir estas variables en su panel, sin archivo.

Es una buena práctica **centralizar y validar** la configuración en un solo módulo (`config/env.ts`). En lugar de esparcir `process.env.PUERTO` por todo el código, este módulo lee todas las variables una vez, les aplica valores por defecto sensatos, verifica que las obligatorias estén presentes (y falla rápido al arrancar si falta alguna), y exporta un objeto `config` tipado. Así, el resto del código importa `config` y obtiene valores limpios, validados y con autocompletado de TypeScript.

Acompaña tu `.env` con un archivo `.env.example` que **sí** subes a Git: tiene las mismas claves pero sin valores reales (o con valores de ejemplo). Sirve de plantilla y documentación: cuando alguien clona tu proyecto, copia `.env.example` a `.env` y rellena sus valores. Esta convención resuelve el

problema de "¿qué variables necesita este proyecto?" sin exponer ningún secreto. Es el estándar en proyectos profesionales.



EJEMPLO PRÁCTICO



Caso: Configurar dotenv, crear el archivo `.env` y centralizar la configuración validada.

TERMINAL

```
# Instala dotenv
$ npm install dotenv
```

`.env`

```
Puerto donde corre el servidor
PORT=3000

Entorno de ejecucion (development | production | test)
NODE_ENV=development

Clave de API secreta (ejemplo)
API_KEY=mi-clave-super-secreta-12345

(En lecciones futuras) URL de la base de datos
DATABASE_URL=postgres://usuario:pass@localhost:5432/tareas
```

`.env.example`

```
Plantilla SIN secretos reales esta S se sube a Git
PORT=3000
NODE_ENV=development
API_KEY=
DATABASE_URL=
```

`src/config/env.ts`

```
import dotenv from 'dotenv';

// Carga las variables del archivo .env en process.env
dotenv.config();
```

```
// Helper: Lee una variable obligatoria o falla al arrancar
function requerida(clave: string): string {
  const valor = process.env[clave];
  if (!valor) {
    throw new Error(`❌ Falta la variable de entorno: ${clave}`);
  }
  return valor;
}

// Configuración centralizada, validada y tipada
export const config = {
  port: Number(process.env.PORT) || 3000,
  nodeEnv: process.env.NODE_ENV || 'development',
  apiKey: requerida('API_KEY'),
  esProduccion: process.env.NODE_ENV === 'production',
};
```

src/server.ts

```
import app from './app';
import { config } from './config/env';

// Usa el puerto de la configuración (no un número hardcodeado)
app.listen(config.port, () => {
  console.log(`🚀 Servidor en http://localhost:${config.port}`);
  console.log(` Entorno: ${config.nodeEnv}`);
});
```

⚠️ EL .env NUNCA VA A GIT

Confirma que `.env` esté en tu `.gitignore`. Si subes un `.env` con secretos reales a un repositorio, considéralos comprometidos: cámbialos de inmediato. El historial de Git es permanente. Sube solo `.env.example` con valores vacíos o de ejemplo.

★ TIPS DEL EXPERTO

- ✦ Carga `dotenv.config()` lo antes posible, al inicio de tu punto de entrada o del módulo de config, antes de leer cualquier `process.env`.
- ✦ **Valida las variables obligatorias al arrancar.** Es mejor que la app falle inmediatamente con un mensaje claro que descubrir a mitad de ejecución que falta una clave.

- ✦ Recuerda que `process.env` siempre devuelve **strings** (o undefined). Convierte explícitamente: `Number(process.env.PORT)` para números, comparaciones para booleanos.
- ✦ Mantén el `.env.example` actualizado: cada vez que agregues una variable nueva a `.env`, añádela también al ejemplo. Es la documentación de configuración de tu proyecto.

PRACTICA LO APRENDIDO



RETO 1

Configura tu proyecto

Instala `dotenv`, crea los archivos `.env` y `.env.example`, y el módulo `config/env.ts`. Modifica `server.ts` para usar `config.port`. Verifica que el servidor lea el puerto del `.env` cambiándolo a 4000 y comprobando que arranca en ese puerto. Asegúrate de que `.env` esté en `.gitignore`.



RETO 2

API Key desde configuración

Conecta el middleware `verificarApiKey` (del reto de la lección 10) con la configuración: en vez de comparar contra el texto hardcodeado `'secreto123'`, compáralo contra `config.apiKey`. Así la clave vive en el `.env` y no en el código. Pruébalo enviando la clave correcta y una incorrecta.



LECCIÓN 16

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

PERSISTENCIA DE DATOS

Guardar los datos para que sobrevivan al reinicio del servidor CONCEPTO

Hasta ahora las tareas vivían en un **array en memoria**: rápido y simple, pero se borran al reiniciar el servidor. La **persistencia** consiste en guardar los datos en un medio durable: un archivo, o mejor, una **base de datos**. Gracias a la arquitectura por capas, solo tenemos que cambiar la **capa de servicio**: los controladores y rutas no se enteran. En esta lección verás cómo persistir en un archivo JSON (simple, para aprender) y las nociones para conectar una base de datos real.

 EXPLICACIÓN DETALLADA

El almacenamiento en memoria es perfecto para desarrollar y aprender, pero tiene un límite fatal: **los datos no sobreviven al reinicio**. Cada vez que el servidor se reinicia (un despliegue, un crash, un cambio de código en desarrollo), todas las tareas creadas desaparecen. Para una aplicación real, los datos deben **persistir**: quedar guardados en un medio que sobreviva al proceso. Las dos opciones principales son archivos y bases de datos.

La opción más simple es **persistir en un archivo JSON**: cada vez que cambian los datos, los escribes a un archivo `tareas.json` en disco, y al arrancar los lees de vuelta. Es fácil de entender y no requiere instalar nada. Tiene limitaciones serias para producción (no maneja bien accesos concurrentes, no escala, las búsquedas son lentas), pero es ideal para aprender el concepto de persistencia y para prototipos pequeños. Usaremos las funciones `fs/promises` de Node para leer y escribir.

Para aplicaciones reales se usa una **base de datos**. Las hay de dos tipos principales: **relacionales** (PostgreSQL, MySQL, SQLite), que organizan datos en tablas con relaciones y usan SQL; y **NoSQL** (MongoDB), que guardan documentos flexibles tipo JSON. La elección depende del proyecto, pero PostgreSQL es una opción excelente y gratuita para la mayoría de casos. Te conectas usando un **ORM** (Object-Relational Mapper) como **Prisma** o **TypeORM**, que te permite trabajar con objetos de TypeScript en lugar de escribir SQL a mano.

Lo más valioso de nuestra arquitectura se revela aquí: como toda la lógica de datos está aislada en `tareas.service.ts`, **migrar de memoria a archivo, o de archivo a base de datos, solo afecta a ese archivo**. Las funciones del servicio (`obtenerTodas`, `crear`, etc.) mantienen la misma firma; cambia solo su implementación interna. Los controladores, las rutas, la validación y el manejo de

errores siguen funcionando sin cambios. Esto es exactamente por lo que separamos en capas desde el principio.



EJEMPLO PRÁCTICO



Caso: Reescribir el servicio para persistir las tareas en un archivo JSON.



src/services/tareas.service.ts

```
import { promises as fs } from 'fs';
import path from 'path';
import { Tarea, CrearTareaDTO, ActualizarTareaDTO } from '../models/tarea.model';

// Ruta del archivo donde se guardan las tareas
const ARCHIVO = path.join(__dirname, '../../data/tareas.json');

// Leer todas las tareas del archivo
async function leerArchivo(): Promise<Tarea[]> {
  try {
    const contenido = await fs.readFile(ARCHIVO, 'utf-8');
    return JSON.parse(contenido);
  } catch {
    return []; // si el archivo no existe aún, empieza vacío
  }
}

// Guardar todas las tareas en el archivo
async function guardarArchivo(tareas: Tarea[]): Promise<void> {
  await fs.mkdir(path.dirname(ARCHIVO), { recursive: true });
  await fs.writeFile(ARCHIVO, JSON.stringify(tareas, null, 2));
}

export async function obtenerTodas(): Promise<Tarea[]> {
  return leerArchivo();
}

export async function obtenerPorId(id: number): Promise<Tarea | undefined> {
  const tareas = await leerArchivo();
  return tareas.find((t) => t.id === id);
}

export async function crear(datos: CrearTareaDTO): Promise<Tarea> {
  const tareas = await leerArchivo();
  const nuevoId = tareas.length > 0
    ? Math.max(...tareas.map((t) => t.id)) + 1
    : 1;

  const nueva: Tarea = {
    id: nuevoId,
    titulo: datos.titulo,
```

```

    completada: false,
    creadaEn: new Date().toISOString(),
  });
  tareas.push(nueva);
  await guardarArchivo(tareas);
  return nueva;
}

export async function actualizar(
  id: number, datos: ActualizarTareaDTO
): Promise<Tarea | undefined> {
  const tareas = await leerArchivo();
  const tarea = tareas.find((t) => t.id === id);
  if (!tarea) return undefined;

  if (datos.titulo !== undefined) tarea.titulo = datos.titulo;
  if (datos.completada !== undefined) tarea.completada = datos.completada;
  await guardarArchivo(tareas);
  return tarea;
}

export async function eliminar(id: number): Promise<boolean> {
  const tareas = await leerArchivo();
  const filtradas = tareas.filter((t) => t.id !== id);
  if (filtradas.length === tareas.length) return false;
  await guardarArchivo(filtradas);
  return true;
}

```

✓ LOS CONTROLADORES AHORA USAN AWAIT

Como el servicio ahora es asíncrono (lee y escribe archivos), los controladores deben usar `await` al llamarlo: `const tareas = await tareasService.obtenerTodas();`. Aquí brilla el `asyncHandler` de la lección 14: envuelve los controladores `async` y captura errores automáticamente. Recuerda añadir `await` en cada llamada al servicio.

★ TIPS DEL EXPERTO

- ✦ Añade la carpeta `data/` (donde se guarda el JSON) a tu `.gitignore` si contiene datos de prueba que no quieres versionar.
- ✦ El archivo JSON sirve para aprender y prototipos. Para producción real, usa una **base de datos** con un ORM como **Prisma**: maneja concurrencia, escala y es más seguro.

- ✦ Gracias a las capas, migrar a una base de datos solo cambia `tareas.service.ts`. Esa es la recompensa de haber separado responsabilidades desde el inicio.
- ✦ `JSON.stringify(datos, null, 2)` guarda el JSON "bonito" (indentado con 2 espacios). Facilita leerlo y depurarlo durante el desarrollo.

PRACTICA LO APRENDIDO



RETO 1

Migra a archivo JSON

Reemplaza el servicio en memoria por la versión con persistencia en archivo. Actualiza todos los controladores para que usen `await` al llamar al servicio. Crea varias tareas, **reinicia el servidor** y verifica que las tareas siguen ahí. Abre el archivo `data/tareas.json` y observa cómo se guardan.



RETO 2

Investiga Prisma

Investiga el ORM **Prisma** (prisma.io). Anota los pasos generales para usarlo: instalar, definir el esquema (`schema.prisma`) con el modelo Tarea, generar el cliente, y cómo se verían las operaciones (`prisma.tarea.findMany()`, `prisma.tarea.create(...)`). No necesitas implementarlo: el objetivo es que entiendas cómo sería el siguiente paso hacia una base de datos real.



LECCIÓN 17

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

PROBAR LA API

Verificar cada endpoint con curl, Thunder Client y Postman

CONCEPTO

Antes de automatizar pruebas, hay que saber **probar a mano** cada endpoint para confirmar que funciona. Las tres herramientas más usadas son: **curl** (línea de comandos, ideal para pruebas rápidas y scripts), **Postman** (aplicación gráfica completa, estándar de la industria) y **Thunder Client** (extensión de VS Code, ligera y cómoda porque vive en tu editor). Con cualquiera de ellas envías peticiones a tu API y verificas el código de estado y el cuerpo de la respuesta.



EXPLICACIÓN DETALLADA

curl es una herramienta de línea de comandos presente en casi todos los sistemas. Su sintaxis básica es `curl URL` para un GET simple. Para otros métodos usas `-X (-X POST)`, para enviar headers `-H`, para el cuerpo `-d`, y para ver los headers de respuesta (incluyendo el código de estado) `-i`. Es perfecta para pruebas rápidas, para incluir en scripts, y porque al ser texto puedes copiar y compartir comandos fácilmente. Vale la pena dominar sus opciones básicas.

Postman es una aplicación de escritorio dedicada a probar APIs. Su interfaz gráfica facilita construir peticiones complejas: seleccionas el método en un menú, escribes la URL, agregas headers y cuerpo en pestañas, y ves la respuesta formateada. Permite **guardar peticiones en colecciones**, definir **variables de entorno** (para alternar entre tu API local y la de producción), y hasta escribir tests automáticos. Es el estándar de facto para equipos que desarrollan y documentan APIs.

Thunder Client es una extensión de VS Code que ofrece una experiencia similar a Postman pero **dentro de tu editor**. Su ventaja es la comodidad: no cambias de aplicación, las peticiones se guardan junto a tu proyecto, y es muy ligera. Para desarrollo individual o equipos pequeños es una opción excelente. Se instala desde el marketplace de extensiones de VS Code y aparece como un ícono de rayo en la barra lateral.

Independientemente de la herramienta, una **buena sesión de pruebas** cubre tanto los casos felices como los de error. Para cada endpoint verifica: que la operación exitosa devuelva el código correcto (200, 201, 204) y los datos esperados; que pedir un recurso inexistente devuelva 404; que enviar datos inválidos devuelva 400. Probar solo el camino feliz es la causa número uno de bugs que aparecen en producción. Sé sistemático: prueba el CRUD completo y sus casos de error.

EJEMPLO PRÁCTICO

 **Caso:** Probar el CRUD completo de la API de tareas con curl, incluyendo casos de error.

TERMINAL

```
# Asegúrate de tener el servidor corriendo: npm run dev

# === CASOS EXITOSOS ===

# Listar todas las tareas
$ curl localhost:3000/api/tareas

# Crear una tarea (POST con cuerpo JSON)
$ curl -X POST localhost:3000/api/tareas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Aprender testing"}'

# Obtener la tarea recién creada
$ curl localhost:3000/api/tareas/3

# Actualizar parcialmente (PATCH)
$ curl -X PATCH localhost:3000/api/tareas/3 \
  -H "Content-Type: application/json" \
  -d '{"completada":true}'

# Borrar (DELETE) y ver el código con -i
$ curl -X DELETE localhost:3000/api/tareas/3 -i

# === CASOS DE ERROR ===

# Tarea inexistente → 404
$ curl localhost:3000/api/tareas/9999 -i

# Datos inválidos → 400
$ curl -X POST localhost:3000/api/tareas \
  -H "Content-Type: application/json" -d '{} ' -i
```

OPCIONES DE CURL MÁS USADAS

Opción	Qué hace	Ejemplo
-X	Define el método HTTP	-X POST
-H	Agrega un header	-H "...: ..."
-d	Envía datos en el cuerpo	-d '{...}'
-i	Muestra headers y código	curl URL -i
-s	Modo silencioso	curl -s URL

★ TIPS DEL EXPERTO

- ✦ Para POST/PUT/PATCH, **siempre** incluye el header `-H "Content-Type: application/json"`. Sin él, Express no parseará el cuerpo y `req.body` llegará vacío.
- ✦ Usa `-i` cuando quieras ver el **código de estado**. Sin él, curl solo muestra el cuerpo y no sabrás si fue 200, 201 o 404.
- ✦ En Postman y Thunder Client, **guarda tus peticiones en una colección**. Tener el CRUD completo guardado te ahorra reescribirlo cada vez que pruebas.
- ✦ Prueba siempre los **casos de error** (404, 400), no solo los exitosos. Los bugs de producción casi siempre están en los caminos que nadie probó.

🚀 PRACTICA LO APRENDIDO



RETO 1

Colección de pruebas completa

Instala Thunder Client (o Postman) y crea una colección "API Tareas" con una petición guardada para cada operación: listar, obtener uno, crear, actualizar (PUT), modificar (PATCH), borrar. Incluye también dos peticiones de error: obtener un id inexistente y crear sin título. Ejecútalas todas y verifica los códigos de estado.



RETO 2

Script de humo (smoke test)

Escribe un archivo `pruebas.sh` con una secuencia de comandos curl que: cree una tarea, la liste, la actualice, la borre, y finalmente intente obtenerla (debería dar 404). Ejecútalo con `bash pruebas.sh`. Este "smoke test" manual te permite verificar todo el flujo con un solo comando.



LECCIÓN 18

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

TESTS AUTOMATIZADOS

Pruebas que se ejecutan solas con Jest y Supertest CONCEPTO

Los **tests automatizados** son código que prueba tu código: verifican que la API se comporte como debe, de forma repetible y sin intervención manual. Usaremos **Jest** (el framework de testing más popular de JavaScript) junto con **Supertest** (una librería que envía peticiones HTTP a tu app Express sin levantar un servidor real). Con ellos escribes pruebas que, con un solo comando, verifican todo tu CRUD en segundos. Si algo se rompe al cambiar el código, los tests te avisan al instante.

 EXPLICACIÓN DETALLADA

Probar a mano (lección 17) está bien al desarrollar, pero no escala: cada vez que cambias algo tendrías que volver a probar todo manualmente, y es fácil olvidar casos. Los tests automatizados resuelven esto: los escribes una vez y se ejecutan en segundos cuantas veces quieras. Su mayor valor aparece al **modificar código existente**: si tus cambios rompen algo que antes funcionaba, los tests fallan inmediatamente, antes de que el bug llegue a producción. Son una red de seguridad.

Jest es el framework que organiza y ejecuta los tests. Provee funciones como `describe` (agrupa tests relacionados), `it` o `test` (define un test individual), y `expect` (hace las afirmaciones: `expect(resultado).toBe(esperado)`). Jest encuentra automáticamente los archivos de test (los que terminan en `.test.ts` o `.spec.ts`), los ejecuta y reporta cuáles pasaron y cuáles fallaron, con detalles de qué se esperaba y qué se obtuvo.

Supertest se especializa en probar APIs HTTP. Le pasas tu app de Express y te permite hacer peticiones (`request(app).get('/api/tareas')`) y verificar la respuesta (`.expect(200)`) sin levantar un servidor en un puerto real: Supertest maneja todo internamente. Esto hace los tests rápidos y aislados. Por eso separamos `app.ts` (la configuración de Express) de `server.ts` (el `listen`): los tests importan `app` directamente, sin arrancar el servidor.

Una buena suite de tests para una API cubre cada endpoint en sus casos principales: que GET devuelva la lista con 200, que POST cree con 201, que GET de un id inexistente dé 404, que POST con datos inválidos dé 400, etc. Estos tests se convierten en **documentación ejecutable** de cómo debe comportarse tu API, y en un contrato que protege contra regresiones. Integrados en un pipeline de CI (Integración Continua), se ejecutan automáticamente en cada cambio antes de desplegar.

EJEMPLO PRÁCTICO

 **Caso:** Instalar Jest y Supertest, configurarlos y escribir tests del CRUD de tareas.

TERMINAL

```
# Instala Jest, Supertest, ts-jest y los tipos (todo de desarrollo)
$ npm install -D jest ts-jest @types/jest supertest @types/supertest

# Genera la configuración de Jest para TypeScript
$ npx ts-jest config:init

# Ejecuta los tests (ya tienes el script "test": "jest")
$ npm test
```

jest.config.js

```
/** @type {import('ts-jest').JestConfigWithTsJest} */
module.exports = {
  preset: 'ts-jest',
  testEnvironment: 'node',
  // Busca archivos *.test.ts dentro de la carpeta tests/
  testMatch: ['**/tests/**/*.test.ts'],
};
```

tests/tareas.test.ts

```
import request from 'supertest';
import app from '../src/app';

// Agrupa todos los tests del recurso tareas
describe('API de Tareas', () => {

  // --- READ: Listar todas ---
  it('GET /api/tareas devuelve 200 y un array', async () => {
    const res = await request(app).get('/api/tareas');
    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });

  // --- CREATE ---
  it('POST /api/tareas crea una tarea y devuelve 201', async () => {
    const res = await request(app)
      .post('/api/tareas')
      .send({ titulo: 'Tarea de prueba' });

    expect(res.status).toBe(201);
    expect(res.body).toHaveProperty('id');
```

```

    expect(res.body.titulo).toBe('Tarea de prueba');
    expect(res.body.completada).toBe(false);
  });

  // --- CREATE con datos inválidos ---
  it('POST sin titulo devuelve 400', async () => {
    const res = await request(app)
      .post('/api/tareas')
      .send({});
    expect(res.status).toBe(400);
  });

  // --- READ: una inexistente ---
  it('GET de un id inexistente devuelve 404', async () => {
    const res = await request(app).get('/api/tareas/99999');
    expect(res.status).toBe(404);
  });

  // --- Flujo completo: crear, actualizar y borrar ---
  it('flujo completo CRUD funciona', async () => {
    // Crear
    const creada = await request(app)
      .post('/api/tareas')
      .send({ titulo: 'Ciclo de vida' });
    const id = creada.body.id;

    // Actualizar
    const actualizada = await request(app)
      .patch(`/api/tareas/${id}`)
      .send({ completada: true });
    expect(actualizada.status).toBe(200);
    expect(actualizada.body.completada).toBe(true);

    // Borrar
    const borrada = await request(app).delete(`/api/tareas/${id}`);
    expect(borrada.status).toBe(204);

    // Verificar que ya no existe
    const buscar = await request(app).get(`/api/tareas/${id}`);
    expect(buscar.status).toBe(404);
  });
});

```

▶ SALIDA EN CONSOLA

PASS tests/tareas.test.ts

API de Tareas

- ✓ GET /api/tareas devuelve 200 y un array (45 ms)
- ✓ POST /api/tareas crea una tarea y devuelve 201 (12 ms)
- ✓ POST sin titulo devuelve 400 (8 ms)

- ✓ GET de un id inexistente devuelve 404 (6 ms)
- ✓ flujo completo CRUD funciona (20 ms)

```
Test Suites: 1 passed, 1 total
Tests:      5 passed, 5 total
Time:       2.1 s
```

✓ POR QUÉ SEPARAMOS `app.ts` DE `server.ts`

Supertest necesita importar tu app de Express **sin** que esté escuchando en un puerto. Por eso `app.ts` solo configura y exporta `app`, mientras que `server.ts` hace el `app.listen(...)`. Los tests importan `app` directamente; el servidor real importa `app` y lo pone a escuchar. Esta separación, que hicimos en la lección 8, ahora rinde frutos.

★ TIPS DEL EXPERTO

- ✦ Nombra tus tests de forma **descriptiva**: "POST sin título devuelve 400" dice exactamente qué verifica. Cuando falla, sabes al instante qué se rompió.
- ✦ Cubre el **camino feliz y los errores**: para cada endpoint, prueba el éxito, el 404 y el 400. Esos tres casos atrapan la mayoría de los bugs.
- ✦ Ejecuta los tests en **modo watch** durante el desarrollo: `npx jest --watch` los re-ejecuta cada vez que guardas. Feedback inmediato.
- ✦ Integra `npm test` en tu flujo: córrelo antes de cada commit y configúralo en CI (GitHub Actions) para que se ejecute automáticamente antes de desplegar.

🚀 PRACTICA LO APRENDIDO



RETO 1

Amplía la cobertura

Agrega tests para los casos que faltan: que PUT actualice correctamente (200), que DELETE de un id inexistente devuelva 404, y que al crear una tarea con título de más de 100 caracteres devuelva 400. Ejecuta `npm test` y verifica que todos pasen. Intenta llegar a 8 o más tests.



Test que detecta un bug

RETO 2

Introduce deliberadamente un bug en tu código (por ejemplo, que `crear` devuelva código 200 en vez de 201). Ejecuta los tests y observa cómo Jest detecta el fallo, mostrándote qué esperaba (201) y qué obtuvo (200). Corrige el bug y comprueba que los tests vuelvan a pasar. Así se siente trabajar con una red de seguridad.



LECCIÓN 19

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

BUILD Y PREPARACIÓN PARA PRODUCCIÓN

Compilar TypeScript a JavaScript y optimizar para desplegar

CONCEPTO

En desarrollo ejecutamos TypeScript directamente con `ts-node-dev`, pero en **producción** se ejecuta JavaScript compilado, que es más rápido y no requiere las herramientas de desarrollo. El proceso de **build** consiste en compilar todo el TypeScript de `src/` a JavaScript en `dist/` con el comando `tsc`, y luego ejecutar ese JavaScript con `node`. En esta lección prepararás tu aplicación para producción: el build, los scripts correctos, las variables de entorno de producción y consideraciones de seguridad.



EXPLICACIÓN DETALLADA

La diferencia entre desarrollo y producción es importante. En **desarrollo** priorizas la comodidad: recarga automática, mensajes de error detallados, ejecución directa de TypeScript. En **producción** priorizas el rendimiento y la estabilidad: código JavaScript ya compilado (sin el costo de transpilar en cada arranque), sin herramientas de desarrollo, con manejo de errores que no expone detalles internos. El comando `npm run build` (que ejecuta `tsc`) produce la versión lista para producción en la carpeta `dist/`.

Cuando ejecutas `tsc`, el compilador lee tu `tsconfig.json`, toma todo el TypeScript de `rootDir` (`src/`) y genera el JavaScript equivalente en `outDir` (`dist/`), respetando la estructura de carpetas. El resultado es código JavaScript estándar que cualquier Node.js puede ejecutar, sin necesidad de TypeScript instalado. Por eso en producción solo necesitas las **dependencies** (como Express), no las **devDependencies** (TypeScript, Jest): el código ya está compilado.

Los **scripts de `package.json`** orquestan esto. `build` ejecuta `tsc` para compilar. `start` ejecuta `node dist/server.js`, el punto de entrada compilado. En el servidor de producción, el flujo típico es: instalar dependencias (`npm install`), compilar (`npm run build`), y arrancar (`npm start`). Algunos proveedores hacen esto automáticamente; otros requieren que lo configures. Lo importante es que `start` apunte al archivo JavaScript compilado, no al TypeScript fuente.

Para producción conviene **añadir robustez**: un endpoint de **health check** (`/health`) que los servicios de hosting consultan para saber si tu app está viva; configurar **CORS** restringido a los dominios que realmente usarán tu API (no `*`); asegurarte de que el manejador de errores **no**

exponga stack traces al cliente; y usar la variable `NODE_ENV=production` para que las librerías activen sus optimizaciones. Opcionalmente, un **Dockerfile** empaqueta tu app en un contenedor reproducible, útil para despliegues consistentes.



EJEMPLO PRÁCTICO



Caso: Compilar el proyecto, verificar el build y preparar los scripts de producción.

TERMINAL

```
# 1. Compila TypeScript → JavaScript (genera la carpeta dist/)
$ npm run build

# 2. Observa el resultado compilado
$ ls dist/
app.js  server.js  config/  controllers/  routes/  services/

# 3. Ejecuta la versión de producción
$ npm start
🚀 Servidor en http://localhost:3000
   Entorno: production

# 4. En producción, las dependencias se instalan sin las de desarrollo
$ npm install --omit=dev
```

package.json (scripts)

```
{
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/server.ts",
    "build": "tsc",
    "start": "node dist/server.js",
    "test": "jest"
  }
}
```

+ ENDPOINT DE HEALTH CHECK

src/app.ts

```
import express from 'express';
import cors from 'cors';
import tareasRouter from './routes/tareas.routes';
```

```
import { errorHandler } from './middlewares/errorHandler';
import { config } from './config/env';

const app = express();

// CORS: en producción, restringe a tus dominios de confianza
app.use(cors({
  origin: config.esProduccion
    ? ['https://mi-frontend.com']
    : '*',
}));
app.use(express.json());

// Health check: Los servicios de hosting lo consultan
app.get('/health', (req, res) => {
  res.json({ estado: 'ok', entorno: config.nodeEnv });
});

app.use('/api/tareas', tareasRouter);

app.use((req, res) => {
  res.status(404).json({ error: 'Ruta no encontrada' });
});
app.use(errorHandler);

export default app;
```

DOCKERFILE OPCIONAL (MULTI-STAGE)

Dockerfile

```
Etapa 1: compilar TypeScript
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

Etapa 2: imagen final, ligera (solo lo necesario)
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY --from=builder /app/dist ./dist
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

⚠ NO OLVIDES EL `.dockerignore`

Si usas Docker, crea un `.dockerignore` con `node_modules`, `dist`, `.env` y `tests`. Evita copiar archivos innecesarios o secretos a la imagen, manteniéndola ligera y segura. Es el equivalente del `.gitignore` para Docker.

★ TIPS DEL EXPERTO

- ✦ **Prueba el build localmente** antes de desplegar: ejecuta `npm run build && npm start` y verifica que la versión compilada funcione igual que en desarrollo.
- ✦ En producción, instala solo dependencias de producción con `npm install --omit=dev` (antes `--production`). Reduce el tamaño y la superficie de ataque.
- ✦ Configura **CORS restringido** en producción: especifica exactamente qué dominios pueden consumir tu API, en lugar de permitir todos con `*`.
- ✦ Agrega siempre un endpoint `/health`. Los servicios de hosting lo usan para reiniciar tu app si deja de responder, y para verificar despliegues.

🚀 PRACTICA LO APRENDIDO



RETO 1

Compila y ejecuta en producción

Ejecuta `npm run build` y examina la carpeta `dist/`: verás tu código en JavaScript. Cambia `NODE_ENV` a `production` en tu `.env`, ejecuta `npm start` y verifica que el servidor arranque con la versión compilada. Prueba el endpoint `/health` y confirma que devuelve el entorno correcto.



RETO 2

Endurece para producción

Revisa tu manejador de errores y asegúrate de que en producción **nunca** devuelva el mensaje técnico de un error 500 (solo "Error interno del servidor"). Haz que, cuando `config.esProduccion` sea `true`, registre el error completo con `console.error` pero responda un mensaje genérico. Verifica que en desarrollo sí puedas ver los detalles para depurar.



LECCIÓN 20

TUTORIAL BACKEND — NODE.JS · EXPRESS · TYPESCRIPT

DESPLEGAR EN PRODUCCIÓN

Publicar tu API en internet para que el mundo la use

CONCEPTO

Desplegar (deploy) es poner tu aplicación en un servidor accesible por internet, con una URL pública. Hoy existen plataformas que hacen esto muy sencillo: conectas tu repositorio de Git, ellas instalan, compilan y ejecutan tu app automáticamente, y te dan una URL con HTTPS. Las más populares para Node.js son **Render**, **Railway** y **Fly.io** (todas con planes gratuitos para empezar). En esta lección desplegarás tu API paso a paso en Render, y conocerás las alternativas y las consideraciones finales de producción.



EXPLICACIÓN DETALLADA

El primer paso para desplegar es tener tu código en un **repositorio Git** (GitHub, GitLab). Las plataformas de despliegue se conectan a tu repositorio y, cada vez que haces **push**, **redespliegan automáticamente** la nueva versión. Este flujo —programas, haces commit, push, y tu cambio está en producción en minutos— es el estándar moderno (Continuous Deployment). Asegúrate de que tu **.gitignore** excluya **node_modules**, **dist** y **.env**: nunca subas dependencias compiladas ni secretos.

Render es una de las plataformas más fáciles para empezar. El proceso: creas una cuenta, eliges "New Web Service", conectas tu repositorio de GitHub, y configuras dos comandos. El **Build Command** (`npm install && npm run build`) instala dependencias y compila TypeScript. El **Start Command** (`npm start`) arranca el servidor con el JavaScript compilado. Render detecta el puerto, te asigna una URL con HTTPS automático, y cada push redespliega. Sin servidores que administrar, sin configurar nginx, sin certificados SSL manuales.

Las **variables de entorno** de producción se configuran en el **panel** de la plataforma, no en un archivo **.env** (que no se sube). En Render, en la sección "Environment" de tu servicio, agregas cada variable (`NODE_ENV=production`, `API_KEY=...`, etc.) con sus valores reales de producción. La plataforma las inyecta en `process.env` al ejecutar tu app, igual que dotenv lo hacía localmente. Un detalle importante: muchos proveedores asignan el puerto mediante la variable **PORT**, así que tu código debe leer `process.env.PORT` (como ya hace tu `config`).

Existen **alternativas** según tus necesidades. **Railway** es similar a Render, muy pulido y rápido de configurar. **Fly.io** despliega contenedores Docker cerca de tus usuarios globalmente. Para más

control, un **VPS** (servidor virtual en DigitalOcean, AWS, etc.) te da una máquina Linux donde instalas todo manualmente —más trabajo, más flexibilidad—. Si empaquetaste tu app con el **Dockerfile** de la lección anterior, puedes desplegarla en cualquier plataforma que soporte contenedores. Para empezar y aprender, Render o Railway con su plan gratuito son ideales.

DESPLEGAR EN RENDER PASO A PASO

TERMINAL

```
# 1. Sube tu proyecto a un repositorio de GitHub
$ git init
$ git add .
$ git commit -m "API de tareas lista para desplegar"
$ git branch -M main
$ git remote add origin https://github.com/tu-usuario/api-tareas.git
$ git push -u origin main

# 2. En render.com:
#   - New + → Web Service
#   - Conecta tu repositorio de GitHub
#   - Configura:
#       Build Command: npm install && npm run build
#       Start Command: npm start
#   - En Environment, agrega tus variables:
#       NODE_ENV = production
#       API_KEY = (tu clave real)
#   - Create Web Service

# 3. Render compila y despliega. Te da una URL como:
#   https://api-tareas.onrender.com

# 4. ¡Prueba tu API en producción!
$ curl https://api-tareas.onrender.com/health
{"estado":"ok","entorno":"production"}

$ curl https://api-tareas.onrender.com/api/tareas
[ ... tus tareas, ahora en internet ... ]
```

COMPARATIVA DE PLATAFORMAS

Plataforma	Ideal para	Plan gratis
Render	Empezar, simplicidad total	Sí
Railway	Rápido, muy pulido	Crédito inicial
Fly.io	Contenedores Docker globales	Sí (limitado)

Plataforma	Ideal para	Plan gratis
VPS	Control total, más trabajo	No (de pago)

✓ CONSIDERACIONES FINALES DE PRODUCCIÓN

Antes de considerar tu API "lista para el mundo real", revisa: (1) **CORS** restringido a tus dominios; (2) los errores 500 **no exponen** detalles internos; (3) los **secretos** están en variables de entorno, nunca en el código; (4) tienes **tests** que se ejecutan antes de desplegar; (5) un endpoint **/health** activo; (6) **logs** para diagnosticar problemas. Con eso tienes una base profesional sólida.

★ TIPS DEL EXPERTO

- ✦ Empieza con **Render** o **Railway** y su plan gratuito. Son lo más rápido para ver tu API viva en internet sin administrar servidores.
- ✦ Configura los **secretos** en el panel de variables de entorno de la plataforma, jamás en el código ni en archivos subidos a Git.
- ✦ Lee siempre el puerto de `process.env.PORT`. La mayoría de plataformas lo asignan dinámicamente; un puerto fijo hardcodeado fallará en producción.
- ✦ Activa los **logs** de tu plataforma y revísalos cuando algo falle. En producción no ves la consola: los logs son tus ojos para diagnosticar problemas.

🚀 PRACTICA LO APRENDIDO



RETO 1

Despliega tu API

Sube tu proyecto a GitHub y desplégalo en Render (o Railway) siguiendo los pasos. Configura las variables de entorno en el panel. Una vez desplegada, prueba tu API en su URL pública con curl o el navegador: el endpoint `/health` y el CRUD de tareas. ¡Habrás puesto tu primer backend en producción!



RETO 2

Documenta y comparte

Crea un archivo `README.md` en tu repositorio que documente tu API: qué hace, cómo instalarla localmente (`npm install`, `npm run dev`), la lista de endpoints con sus métodos y ejemplos, y la URL de producción. Una buena documentación es la

diferencia entre un proyecto que otros pueden usar y uno que solo entiendes tú.
Compártelo en tu portafolio.